ACADEMIC EXAMINATION GUIDE

API & Microservices Architecture

Complete Question and Answer Collection (Questions 1 to 62)

Comprehensive academic revision and examination preparation collection covering Spring Framework, Inversion of Control (IoC), Dependency Injection, Spring Boot, Spring Data JPA, Hibernate, RESTful Web Services, Microservices Architecture, SOAP, and API Security.

SUBJECT / DOMAIN **API & Microservices**	TOTAL QUESTIONS **62 Fully Solved Questions**
CONTENT INTEGRITY **Strict Source-Preserved Content**	FORMAT STANDARDS **Academic Technical Reference**

Authoritative Academic Question & Answer Compilation • Single Source of Truth

Table of Contents (Part 1: Questions 1 to 31)

Spring Core, Dependency Injection, Inversion of Control, Spring Boot, and Data Persistence

Q01 What is the Spring Framework? Page 4
Q02 What is the use of @ComponentScan? Page 5
Q03 What is Autowiring in Spring? Page 6
Q04 Name any Four Spring Bean Scopes Page 7
Q05 Differentiate Between Query Methods and Named … Page 8
Q06 What is Pagination in Spring Data JPA? Page 9
Q07 State Any Two Advantages of Web Services Page 10
Q08 What is the Purpose of the HTTP GET Method in … Page 10
Q09 What is Exception Handling in Spring REST? Page 11
Q10 What is Meant by Securing a REST Endpoint? Page 12
Q11 Define Spring Framework. Page 13
Q12 Explain Why Spring Framework Is Used in Applica… Page 14
Q13 What is IoC (Inversion of Control)? Page 15
Q14 Define Dependency Injection (DI). Page 16
Q15 List Different Spring Framework Modules Page 17
Q16 Explain the Concept of Auto Scanning with Annota… Page 18
Q17 Explain IoC Container Using Java-Based Configur… Page 20
Q18 Differentiate Between Constructor Injection and Se… Page 21
Q19 Write Steps to Create a Spring Boot Hello World A… Page 23
Q20 Apply @ComponentScan to Enable Auto Scanning. Page 24
Q21 What is @SpringBootApplication Annotation? Page 25
Q22 Describe How Autowiring Works Internally in Spring. Page 27
Q23 Create a Test Class to Verify Autowiring Functional… Page 28
Q24 Explain Different Bean Scopes in Spring and Desc… Page 29
Q25 Describe the Role of Logback in Spring Boot. Page 30
Q26 Explain the Concept of AOP and Its Benefits. Page 32
Q27 Develop a Spring Boot Application with Employee … Page 33
Q28 What Are the Major Limitations of the JDBC API? … Page 34
Q29 Compare Derived Query Methods, @Query, and N… Page 36
Q30 Explain Why Transactions Are Required in Spring … Page 37
Q31 Implement Pagination and Sorting for the User Ent… Page 39

Table of Contents (Part 2: Questions 32 to 62)

Web Services, RESTful Architecture, Service-Oriented Architecture (SOA), and API Security

Q32 What Are Web Services? Explain Why Web Servic… Page 40
Q33 Create a REST API to Retrieve User Information U… Page 42
Q34 Explain the Different Types of Web Services. Com… Page 43
Q35 Explain Service-Oriented Architecture (SOA) and It… Page 44
Q36 Why Is REST Widely Used for Modern Web APIs? … Page 46
Q37 Explain the Differences Between `@PathVariable`,… Page 47
Q38 Explain Spring REST and Describe the Steps Invol… Page 49
Q39 Create a REST API to Retrieve User Information U… Page 50
Q40 Explain Data Validation, Exception Handling, COR… Page 52
Q41 Create a Spring Boot REST Application for Managi… Page 53
Q42 Create a Spring Boot RESTful Web Service for a … Page 55
Q43 Apply Java-Based Configuration Techniques to Set… Page 57
Q44 Compare Constructor and Setter Injection with Ex… Page 59
Q45 Discuss About Dependency Injection (DI). Page 60
Q46 Explain About Auto Scanning in Brief. Page 62
Q47 Discuss About Creating a Spring Boot Application. Page 63
Q48 Explain the Scope of a Bean and Logger. Page 64
Q49 Illustrate How to Implement AOP Advices in Spring. Page 66
Q50 Explain About Spring AOP. Page 67
Q51 Explain the Limitations of JDBC API and Query Ap… Page 69
Q52 Demonstrate How to Implement Pagination Using … Page 70
Q53 Explain the Update Operation in Spring Data JPA. Page 71
Q54 Discuss Named Queries and @Query in Spring D… Page 73
Q55 Illustrate SOA – Service Oriented Architecture. Page 75
Q56 Explain Various Types of Web Services. Page 77
Q57 Show How to Design and Implement RESTful API… Page 78
Q58 Explain SOAP-Based Web Services. Page 81
Q59 Explain `@RequestBody`, `ResponseEntity` and P… Page 83
Q60 Apply Exception Handling Techniques and Data Va… Page 85
Q61 Explain the Steps to Create a REST Client. Page 87
Q62 Discuss About Securing Spring REST Endpoints. Page 88

QUESTION 1

1. What is the Spring Framework?

1. What is the Spring Framework?

Spring Framework is an open-source, lightweight Java framework used to develop enterprise-level applications. It simplifies Java development by providing features like **Dependency Injection (DI), Inversion of Control (IoC), transaction management and database connectivity.**

Main Features

1. **Lightweight:** Requires fewer resources and simplifies application development.
2. **IoC (Inversion of Control):** Spring manages the creation and lifecycle of objects.
3. **DI (Dependency Injection):** Automatically provides required dependencies to objects.
4. **AOP (Aspect-Oriented Programming):** Handles common tasks like logging and security.
5. **Transaction Management:** Simplifies database transaction handling.
6. **Spring MVC:** Helps develop web applications.
7. **Database Integration:** Supports JDBC, Hibernate and JPA.

Spring Framework Architecture

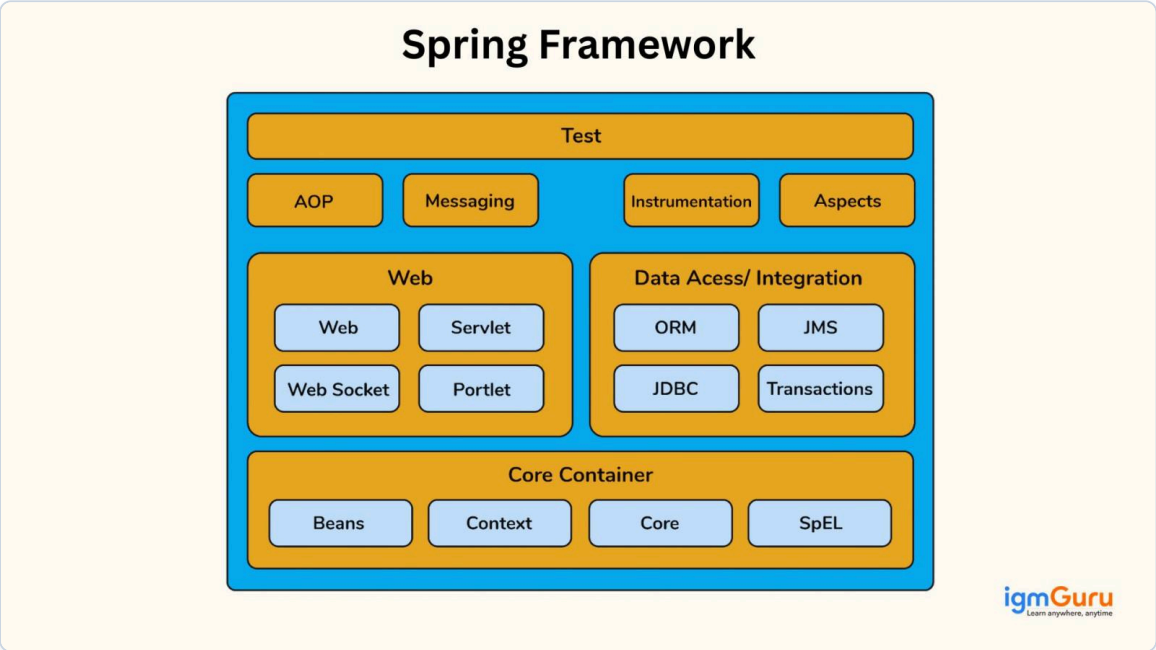


Figure 1.1: Spring Framework Architecture (Core Container, AOP, Data Access, Web, Test)

Example

```
@Component
class Student {
    void display() {
        System.out.println("Student Details");
    }
}
```

Here, `@Component` tells Spring to manage the Student class as a Spring Bean.

Advantages

- Reduces boilerplate code.

- Makes applications loosely coupled.
- Improves code reusability.
- Simplifies testing and maintenance.

Easy Memory Trick

Remember **I-D-A-T-M**:

- **I** – Inversion of Control
- **D** – Dependency Injection
- **A** – Aspect-Oriented Programming
- **T** – Transaction Management
- **M** – MVC

Important: Spring Framework is mainly known for IoC and Dependency Injection.

QUESTION 2

2. What is the use of @ComponentScan?

2. What is the use of @ComponentScan?

`@ComponentScan` is an annotation in the **Spring Framework** used to automatically scan specified packages to identify classes annotated with `@Component`, `@Service`, `@Repository`, and `@Controller`.

It registers the detected classes as **Spring Beans** in the Spring IoC Container. This helps Spring manage objects automatically without requiring manual bean configuration.

Uses of @ComponentScan

1. **Automatic Component Detection:** It searches specified packages for classes containing Spring annotations.
2. **Bean Registration:** It automatically registers detected classes as Spring Beans.
3. **Reduces Manual Configuration:** Developers do not need to manually define every bean in the configuration class.
4. **Package Scanning:** It allows developers to specify which packages Spring should scan.

Example

```
@Configuration
@ComponentScan(basePackages = "com.example")
public class AppConfig {
}
```

Explanation: - `@Configuration` indicates that the class contains Spring configuration. - `@ComponentScan` enables package scanning. - `basePackages = "com.example"` specifies the package that Spring should scan.

Suppose we have the following class:

```
@Component
public class Student {
    public void display() {
        System.out.println("Student Details");
    }
}
```

Spring automatically detects the `Student` class and registers it as a Spring Bean.

Conclusion

`@ComponentScan` simplifies Spring application configuration by automatically detecting and registering annotated classes as Spring Beans.

Important: It is mainly used for automatic component scanning and bean registration.

QUESTION 3

3. What is Auto Wiring in Spring?

3. What is Autowiring in Spring?

Autowiring in Spring is a feature that automatically injects the required dependencies (objects) into a class without manually creating or configuring them.

It is a part of **Dependency Injection (DI)** and helps reduce manual coding and makes applications loosely coupled.

Types of Autowiring

Spring supports different types of dependency injection:

1. **Constructor Injection:** Dependencies are provided through the class constructor.
2. **Setter Injection:** Dependencies are provided using setter methods.
3. **Field Injection:** Dependencies are directly injected into class variables using `@Autowired`.

@Autowired Annotation

Spring uses the `@Autowired` annotation to automatically inject dependencies.

Example:

```
@Component
class Student {
    void display() {
        System.out.println("Student Details");
    }
}

@Component
class College {

    @Autowired
    Student student;

    void show() {
        student.display();
    }
}
```

Explanation:

- `@Component` tells Spring to manage the classes as Spring Beans.
- `@Autowired` tells Spring to inject the required `Student` object into the `College` class.
- Spring automatically manages the dependency through its IoC Container.

Advantages of Autowiring

1. Reduces manual object creation.
2. Simplifies dependency management.
3. Reduces boilerplate code.

4. Promotes loose coupling.
5. Improves code maintainability.

Conclusion: Autowiring simplifies dependency injection by allowing Spring to automatically provide the required objects to a class.

Important: The `@Autowired` annotation is commonly used for automatic dependency injection in Spring.

QUESTION 4

4. Name any four Spring Bean scopes

4. Name any Four Spring Bean Scopes

Spring Bean Scopes define the lifecycle and visibility of a Spring Bean, specifying how many instances Spring creates and how long they remain available.

The four Spring Bean scopes are:

1. **Singleton Scope:**
2. Only one instance of a bean is created per Spring IoC Container.
3. It is the default scope in Spring.
4. **Prototype Scope:**
5. A new bean instance is created every time it is requested from the container.
6. **Request Scope:**
7. A new bean instance is created for each HTTP request.
8. It is used in web applications.
9. **Session Scope:**
10. A single bean instance is created for each HTTP session.
11. It remains available throughout that session.

Example

```
@Component
@Scope("prototype")
public class Student {
}
```

Here, `@Scope("prototype")` tells Spring to create a new `Student` bean instance whenever it is requested from the container.

Note: Spring also supports other scopes, such as Application and WebSocket scopes.

Easy Memory Trick: S-P-R-S = Singleton, Prototype, Request, Session.

QUESTION 5

5. Differentiate between Query Methods and Named Queries

5. Differentiate Between Query Methods and Named Queries

In **Spring Data JPA**, Query Methods and Named Queries are two ways to retrieve data from a database. Both are used to perform database operations, but they differ in how queries are defined and executed.

Difference Between Query Methods and Named Queries

Query Methods	Named Queries
1. Queries are created using method names.	Queries are defined with a specific name.
2. Follow predefined naming conventions.	Use JPQL or SQL query definitions.
3. Written directly in the Repository interface.	Usually defined using `@NamedQuery` or `@NamedNativeQuery`.
4. Suitable for simple queries.	Suitable for more complex queries.
5. Require less coding.	Require separate query definitions.
6. Spring Data JPA derives the query from the method name.	The application refers to the predefined query by its name.

1. Query Methods Example

```
public interface StudentRepository
       extends JpaRepository<Student, Integer> {

    List<Student> findByName(String name);
}
```

Here, Spring Data JPA automatically generates a query based on the method name `findByName()`.

2. Named Queries Example

```
@Entity
@NamedQuery(
    name = "Student.findAllStudents",
    query = "SELECT s FROM Student s"
)
public class Student {
    @Id
    private int id;
}
```

Repository method:

```
List<Student> findAllStudents();
```

Here, the query is defined separately using `@NamedQuery` and can be referenced by its name.

Conclusion: Query Methods are best for simple database queries, whereas Named Queries are useful when queries need to be predefined and reused.

Easy Memory Trick: - Query Methods = Query from Method Name. - Named Queries = Predefined Query with a Name.

QUESTION 6

6. What is Pagination in Spring Data JPA?

6. What is Pagination in Spring Data JPA?

Pagination in Spring Data JPA is a technique used to divide a large amount of database records into smaller pages. Instead of retrieving all records at once, it fetches a limited number of records per page.

It improves application performance and reduces memory usage, especially when working with large datasets.

Main Features of Pagination

1. **Pageable:** An interface used to define the page number and page size.
2. **PageRequest:** A class used to create pagination requests.
3. **Page:** An interface that contains the requested records and information about the total pages and records.
4. **Sorting:** Pagination can also be combined with sorting to arrange records.

Example

```
public interface StudentRepository
        extends JpaRepository<Student, Integer> {
}
```

Using Pagination:

```
Pageable pageable = PageRequest.of(0, 5);

Page<Student> students =
    studentRepository.findAll(pageable);
```

Explanation

- `PageRequest.of(0, 5)` creates a request for the first page containing 5 records.
- `findAll(pageable)` retrieves the requested records.
- `Page<Student>` stores the result and pagination details.

Example:

Suppose a database contains 100 student records and the page size is 10.

- Page 1: Records 1–10
- Page 2: Records 11–20
- Page 3: Records 21–30
- Page 10: Records 91–100

Advantages

1. Reduces unnecessary data retrieval.
2. Improves application performance.
3. Reduces memory consumption.
4. Makes large datasets easier to display.

Conclusion: Pagination helps retrieve database records in smaller pages using `Pageable`, `PageRequest`, and `Page` in Spring Data JPA.

QUESTION 7

7. State any two advantages of Web Services

7. State Any Two Advantages of Web Services

Web Services are software components that allow different applications to communicate and exchange data over a network using standard protocols such as HTTP. They help applications developed using different programming languages and platforms work together.

Two Advantages of Web Services

1. Platform Independence

- Web Services can work on different operating systems and platforms.
- They allow applications developed using different programming languages to communicate.
- They do not depend on a specific operating system or programming language.
- Example: A Java application running on Windows can communicate with a Python application running on Linux using a Web Service.

2. Reusability

- Web Services allow the same functionality to be used by multiple applications.
- Developers do not need to write the same code repeatedly.
- A single Web Service can be accessed by different applications whenever required.
- Example: An online payment service can be reused by multiple shopping applications for processing payments.

Other Advantages of Web Services

Although the question asks for only two advantages, some other important advantages are:

1. **Interoperability:** Different applications can communicate with each other.
2. **Loose Coupling:** Applications can interact without depending heavily on each other's internal implementation.
3. **Easy Integration:** Web Services make it easier to connect different software systems.

Conclusion

Web Services make communication between different applications easier. **Platform Independence** and **Reusability** are two major advantages that help developers build flexible and reusable software applications.

Important Exam Points: Platform Independence and Reusability are the two main advantages asked in this question.

QUESTION 8

8. What is the purpose of the HTTP GET method in REST?

8. What is the Purpose of the HTTP GET Method in REST?

The **HTTP GET method** in REST (Representational State Transfer) is used to **retrieve or fetch data from a server**. It allows clients to request specific resources without modifying or deleting the data stored on the server.

It is one of the commonly used HTTP methods in RESTful Web Services.

Main Purposes of HTTP GET

1. **Retrieve Data:** It fetches information from the server, such as student details, product information or user records.
2. **Access Resources:** It allows clients to access specific resources using a URL.

3. **Read Operations:** It is mainly used for reading data without changing the server's state.
4. **Query Parameters:** It supports query parameters to filter or search for specific information.

Example

Suppose a student management application needs to retrieve student details.

HTTP Request:

```
GET /api/students/101 HTTP/1.1
Host: example.com
```

Response:

```
{
  "id": 101,
  "name": "Rahul",
  "course": "AI and ML"
}
```

Important Characteristics

- **Safe Method:** GET does not modify the server's data.
- **Idempotent:** Repeating the same GET request produces the same intended effect.
- **Cacheable:** GET responses can be cached to improve performance.
- **No Request Body Required:** GET requests generally use URL parameters to send filtering information.

Conclusion

The main purpose of the HTTP GET method in REST is to retrieve resources from a server without modifying the existing data.

Important: GET = Retrieve or Read Data.

QUESTION 9

9. What is Exception Handling in Spring REST?

9. What is Exception Handling in Spring REST?

Exception Handling in Spring REST is a mechanism used to handle errors and exceptions that occur while processing REST API requests. It prevents the application from crashing and provides meaningful error responses to clients.

Spring provides annotations such as `@ExceptionHandler`, `@ControllerAdvice`, and `@RestControllerAdvice` to manage exceptions.

Main Annotations Used

1. **@ExceptionHandler:** Used to handle specific exceptions within a controller.
2. **@ControllerAdvice:** Provides global exception handling across multiple controllers.
3. **@RestControllerAdvice:** Combines `@ControllerAdvice` and `@ResponseBody` to return error responses directly in the response body.

Example

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleException(
            Exception ex) {

        return ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Something went wrong");
    }
}
```

Explanation

- `@RestControllerAdvice` enables global exception handling.
- `@ExceptionHandler(Exception.class)` handles exceptions of type `Exception`.
- `ResponseEntity` is used to return the HTTP status code and error message.
- `INTERNAL_SERVER_ERROR` represents HTTP status code **500**.

Advantages of Exception Handling

1. **Prevents Application Failure:** Handles unexpected errors properly.
2. **Meaningful Error Messages:** Provides understandable error information to clients.
3. **Centralized Handling:** Manages exceptions from multiple controllers in one place.
4. **Better API Response:** Returns appropriate HTTP status codes such as 400, 404 and 500.

Conclusion: Exception Handling in Spring REST helps manage application errors and return proper HTTP error responses to clients.

QUESTION 10

10. What is meant by securing a REST endpoint?

10. What is Meant by Securing a REST Endpoint?

Securing a REST Endpoint means protecting a REST API URL from unauthorized access and ensuring that only authenticated and authorized users can access specific resources or perform operations.

It helps protect sensitive data and prevents unauthorized users from accessing or modifying application information.

Main Methods of Securing REST Endpoints

1. **Authentication:** Verifies the identity of a user before allowing access. Example: Username and password login.
2. **Authorization:** Determines what an authenticated user is allowed to access or perform. Example: Only an admin can delete student records.
3. **HTTPS:** Uses encrypted communication between the client and server to protect data from being intercepted.
4. **Token-Based Authentication:** Uses tokens such as JWT (JSON Web Token) to verify users when they send API requests.
5. **Role-Based Access Control:** Provides permissions based on user roles such as Admin, Teacher and Student.

Example

Consider a Student Management REST API.

```
GET /api/students
Authorization: Bearer <token>
```

Explanation: - The client sends a request to access student details. - The server checks the authentication token. - If the token is valid and the user has permission, access is granted. - Otherwise, the server returns an appropriate error response.

Common HTTP Status Codes

Status Code	Meaning
401	Unauthorized
403	Forbidden
200	Successful Request

Importance

- Protects sensitive information.
- Prevents unauthorized access.
- Ensures secure communication.
- Controls user permissions.

Conclusion: Securing a REST endpoint ensures that API resources are accessed only by authorized users through authentication, authorization and secure communication.

QUESTION 11

11. Define Spring Framework.

11. Define Spring Framework.

Spring Framework is an open-source, lightweight Java framework used to develop enterprise-level applications. It simplifies Java application development by providing features such as **Inversion of Control (IoC), Dependency Injection (DI), Aspect-Oriented Programming (AOP), transaction management and database connectivity.**

It helps developers build applications that are loosely coupled, reusable, maintainable and easier to test.

Main Features of Spring Framework

1. **Lightweight:** Spring provides a lightweight infrastructure for developing Java applications.
2. **Inversion of Control (IoC):** Spring manages the creation and lifecycle of objects instead of developers managing them manually.
3. **Dependency Injection (DI):** Automatically provides the required dependencies to objects.
4. **Aspect-Oriented Programming (AOP):** Separates common tasks such as logging, security and transaction management from business logic.
5. **Transaction Management:** Provides support for managing database transactions.
6. **Spring MVC:** Supports the development of web applications using the Model-View-Controller architecture.
7. **Database Integration:** Supports technologies such as JDBC, Hibernate and JPA.

Example

```
@Component
public class Student {
    public void display() {
        System.out.println("Hello Student");
    }
}
```

Here, `@Component` tells Spring to manage the `Student` class as a Spring Bean.

Conclusion

Spring Framework simplifies Java application development by providing built-in features for object management, dependency injection, database operations and web development.

Important Exam Point: Spring Framework is mainly known for **IoC and Dependency Injection**, which help create loosely coupled applications.

QUESTION 12

12. Explain why Spring Framework is used in application development

12. Explain Why Spring Framework Is Used in Application Development.

Spring Framework is used in application development because it simplifies Java programming and provides tools to build secure, flexible, reusable and maintainable applications. It reduces development complexity through features like Dependency Injection, Inversion of Control and transaction management.

Reasons for Using Spring Framework

1. **Simplifies Development:** Spring reduces boilerplate code and makes application development easier.
2. **Dependency Injection (DI):** It automatically provides required objects to classes, reducing manual object creation.
3. **Loose Coupling:** It reduces dependency between application components, making code easier to modify and maintain.
4. **Database Integration:** It supports JDBC, Hibernate and JPA for database connectivity and operations.
5. **Transaction Management:** It simplifies database transaction handling and helps maintain data consistency.
6. **Web Application Development:** Spring MVC provides features to develop web applications and REST APIs.
7. **Security:** Spring integrates with Spring Security to provide authentication and authorization.
8. **Easy Testing:** It supports unit testing and integration testing of applications.

Example

```
@Component
class Student {
    void display() {
        System.out.println("Student Details");
    }
}
```

Here, Spring manages the `Student` object as a Spring Bean, reducing manual object creation.

Conclusion

Spring Framework is widely used because it simplifies application development, improves code reusability, reduces dependencies and provides support for database operations, web development and security.

Important Exam Point: Spring Framework is mainly used to develop **loosely coupled, reusable, secure and maintainable Java applications**.

QUESTION 13

13. What is IoC (Inversion of Control)?

13. What is IoC (Inversion of Control)?

IoC (Inversion of Control) is a design principle in the Spring Framework in which the control of creating, managing and configuring objects is transferred from the programmer to the Spring Container.

Normally, developers create objects manually using the `new` keyword. But with IoC, Spring manages object creation and their lifecycle automatically.

Working of IoC

1. The developer creates Java classes and defines their dependencies.
2. Spring Container scans or reads the configuration.
3. It creates the required objects (Spring Beans).
4. It manages their lifecycle.
5. It injects dependencies wherever required.

Example

Without IoC:

```
class Student {
    void display() {
        System.out.println("Student Details");
    }
}

class College {
    Student s = new Student();
}
```

Here, the developer manually creates the `Student` object using the `new` keyword.

With IoC:

```
@Component
class Student {
    void display() {
        System.out.println("Student Details");
    }
}

@Component
class College {
    @Autowired
    Student student;
}
```

Here, Spring automatically creates and injects the `Student` object into the `College` class.

Advantages of IoC

1. **Loose Coupling:** Reduces dependency between classes.
2. **Easy Maintenance:** Makes applications easier to modify.
3. **Reusability:** Allows objects to be reused.

4. **Easy Testing:** Makes unit testing easier.

Conclusion

IoC transfers object creation and management responsibility from the developer to the Spring Container.

Important: IoC is the main principle behind Dependency Injection (DI) in Spring.

QUESTION 14

14. Define Dependency Injection.

14. Define Dependency Injection (DI).

Dependency Injection (DI) is a design pattern in the Spring Framework in which the required objects (dependencies) are provided to a class by the Spring Container instead of the class creating those objects manually.

It is a method of implementing **Inversion of Control (IoC)**. DI helps create loosely coupled applications and makes code easier to maintain and test.

Types of Dependency Injection

Spring supports three main types of Dependency Injection:

1. **Constructor Injection:** Dependencies are provided through the class constructor.
2. **Setter Injection:** Dependencies are provided using setter methods.
3. **Field Injection:** Dependencies are directly injected into class variables using the `@Autowired` annotation.

Example

```
@Component
class Student {
    void display() {
        System.out.println("Student Details");
    }
}

@Component
class College {

    @Autowired
    Student student;
}
```

Explanation

- `@Component` tells Spring to manage the classes as Spring Beans.
- `@Autowired` tells Spring to inject the required `Student` object into the `College` class.
- Spring Container automatically manages the dependency.

Advantages of Dependency Injection

1. **Loose Coupling:** Reduces dependency between classes.
2. **Code Reusability:** Allows objects to be reused.
3. **Easy Testing:** Makes unit testing easier.
4. **Easy Maintenance:** Makes applications easier to modify and manage.

Conclusion

Dependency Injection simplifies object management by allowing Spring to provide the required dependencies automatically instead of creating objects manually.

Important: DI is a way to achieve IoC in the Spring Framework.

QUESTION 15

15. List different Spring Framework modulesv

15. List Different Spring Framework Modules

The **Spring Framework** is divided into different modules, and each module provides specific functionality for developing Java applications. These modules help developers manage objects, perform database operations, develop web applications, handle security-related tasks and test applications.

Different Modules of Spring Framework

The major Spring Framework modules are:

1. Core Container Module - It provides the basic functionality of the Spring Framework. - It supports **Inversion of Control (IoC)** and **Dependency Injection (DI)**. - It manages the creation and lifecycle of Spring Beans. - Main parts include Core, Beans, Context and SpEL.

2. AOP Module (Aspect-Oriented Programming) - It separates common tasks from business logic. - It supports logging, security and transaction management. - It improves code reusability and maintainability.

3. Data Access / Integration Module - It provides support for database connectivity and operations. - It supports JDBC, ORM, JPA and transaction management. - It simplifies database interaction.

4. Web Module - It provides features required for web application development. - It supports web-related integration and request handling.

5. Spring MVC Module - It follows the **Model-View-Controller (MVC)** architecture. - It helps develop web applications and RESTful services. - It separates application logic, user interface and request handling.

6. Test Module - It provides support for unit testing and integration testing. - It integrates with testing frameworks such as JUnit. - It helps test Spring applications efficiently.

Spring Framework Architecture

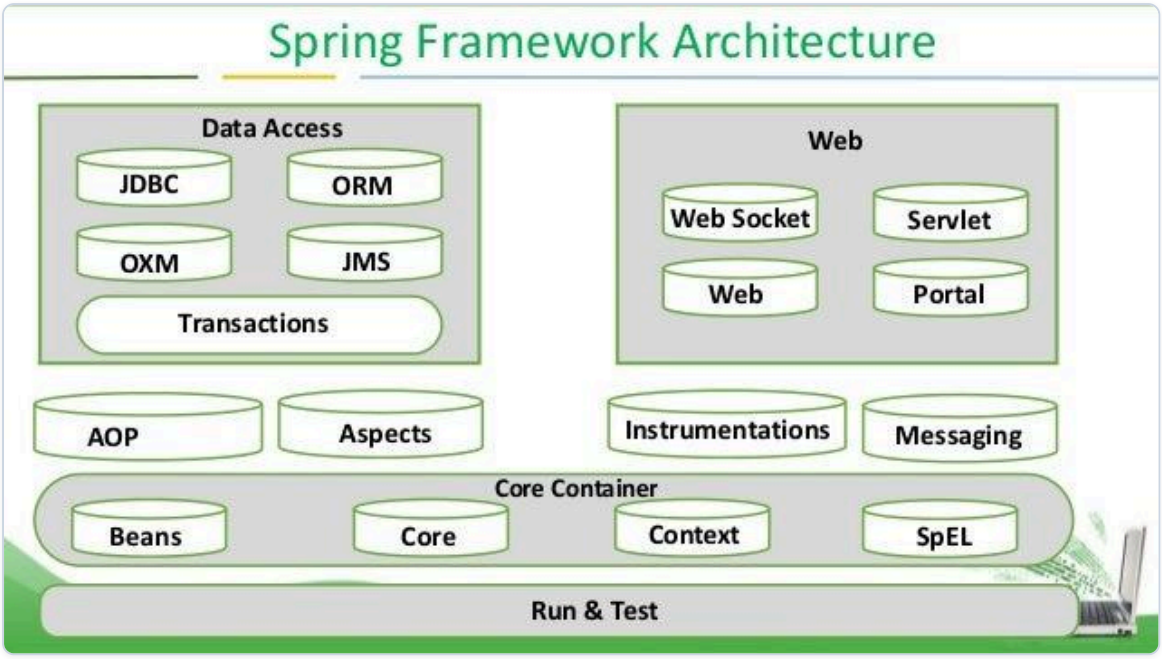


Figure 15.1: Spring Framework Architecture Modules Labeled Diagram

Conclusion

Spring Framework modules provide different functionalities required for Java application development. They work together to make applications flexible, reusable, maintainable and easier to develop.

Important Exam Point: Remember these six major modules:

1. Core Container
2. AOP
3. Data Access / Integration
4. Web
5. Spring MVC
6. Test

QUESTION 16

16. Explain the concept of Auto Scanning with annotations and how

16. Explain the Concept of Auto Scanning with Annotations and How It Works.

Auto Scanning in the Spring Framework is a feature that automatically detects classes annotated with Spring annotations and registers them as **Spring Beans** in the IoC Container. It reduces manual configuration and makes application development easier.

Spring uses the `@ComponentScan` annotation to scan specified packages and identify classes that need to be managed by the Spring Container.

Important Annotations Used in Auto Scanning

1. **@Component:** Marks a Java class as a Spring-managed component.

2. **@Service:** Identifies a class that contains business logic.
3. **@Repository:** Identifies a class responsible for database operations.
4. **@Controller:** Identifies a class that handles web requests.
5. **@RestController:** Identifies a class that handles REST API requests and returns data directly.
6. **@ComponentScan:** Tells Spring which packages to scan for annotated classes.

How Auto Scanning Works

Auto Scanning works through the following steps:

1. **Define Components:** Developers add annotations such as `@Component`, `@Service` or `@Repository` to Java classes.
2. **Enable Component Scanning:** The `@ComponentScan` annotation specifies the packages to scan.
3. **Scan Packages:** Spring searches the specified packages for annotated classes.
4. **Register Beans:** Spring automatically registers the detected classes as Spring Beans.
5. **Dependency Injection:** Spring injects the required dependencies using annotations such as `@Autowired`.

Example

```
@Component
public class Student {
    public void display() {
        System.out.println("Student Details");
    }
}
```

Configuration Class:

```
@Configuration
@ComponentScan(basePackages = "com.example")
public class AppConfig {
}
```

Explanation

- `@Component` marks the Student class as a Spring component.
- `@Configuration` defines the configuration class.
- `@ComponentScan` instructs Spring to scan the `com.example` package.
- Spring detects the Student class and registers it as a Bean.

Advantages of Auto Scanning

1. Reduces manual bean configuration.
2. Simplifies application development.
3. Supports automatic dependency management.
4. Improves code organization and maintainability.

Conclusion: Auto Scanning automatically detects and registers Spring components as Beans using annotations and `@ComponentScan`, reducing manual configuration in Spring applications.

QUESTION 17

Explain IoC container using Java-based configuration.

17. Explain IoC Container Using Java-Based Configuration.

IoC (Inversion of Control) Container is the core part of the Spring Framework that manages the creation, configuration and lifecycle of objects called **Spring Beans**. Instead of creating and managing objects manually, developers allow the Spring Container to handle these responsibilities.

In **Java-Based Configuration**, Spring uses Java classes and annotations to configure beans instead of XML configuration files.

Main Annotations Used

1. **@Configuration:** Indicates that a Java class contains Spring bean configuration.
2. **@Bean:** Defines a method whose returned object is registered as a Spring Bean.
3. **@Autowired:** Automatically injects required dependencies.
4. **@ComponentScan:** Scans specified packages to detect Spring components.

Working of IoC Container

1. The developer creates a Java configuration class.
2. The `@Configuration` annotation identifies the configuration class.
3. The `@Bean` annotation defines the objects managed by Spring.
4. Spring Container reads the configuration and creates the required beans.
5. It manages bean dependencies and their lifecycle.
6. The application retrieves the required beans from the container.

Example

Step 1: Create a Java Class

```
public class Student {
    public void display() {
        System.out.println("Student Details");
    }
}
```

Step 2: Create Configuration Class

```
@Configuration
public class AppConfig {

    @Bean
    public Student student() {
        return new Student();
    }
}
```

Step 3: Access Bean Using IoC Container

```java
public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(
                AppConfig.class);

        Student s = context.getBean(Student.class);

        s.display();
    }
}
```

Explanation

- `@Configuration` defines the configuration class.
- `@Bean` registers the Student object in the Spring Container.
- `AnnotationConfigApplicationContext` creates the IoC Container using Java configuration.
- `getBean()` retrieves the Student object from the container.
- `display()` executes the method.

Advantages

1. Reduces manual object creation.
2. Avoids XML configuration.
3. Supports Dependency Injection.
4. Improves code maintainability.
5. Makes application configuration easier.

Conclusion: Java-Based Configuration allows the Spring IoC Container to manage beans using Java classes and annotations instead of XML files.

QUESTION 18

Differentiate between Constructor Injection and Setter Injection

18. Differentiate Between Constructor Injection and Setter Injection

Constructor Injection and **Setter Injection** are two types of Dependency Injection (DI) in the Spring Framework. Both are used to provide required dependencies to a class, but they differ in how dependencies are supplied.

Difference Between Constructor Injection and Setter Injection

Constructor Injection	Setter Injection
1. Dependencies are provided through a class constructor.	Dependencies are provided through setter methods.
2. Uses a constructor to inject dependencies.	Uses setter methods to inject dependencies.
3. Dependencies are provided when the object is created.	Dependencies can be provided after the object is created.
4. Suitable for mandatory dependencies.	Suitable for optional dependencies.
5. Makes dependencies immutable when final fields are used.	Dependencies can be changed after object creation.
6. Helps ensure that required dependencies are available during object creation.	Allows dependencies to be set or changed later.
7. Generally preferred for required dependencies.	Useful when dependencies are optional or need modification.

1. Constructor Injection Example

```
@Component
class Student {
    private final Address address;

    @Autowired
    Student(Address address) {
        this.address = address;
    }
}
```

Explanation: Spring provides the `Address` object through the constructor when creating the `Student` object.

2. Setter Injection Example

```
@Component
class Student {
    private Address address;

    @Autowired
    public void setAddress(Address address) {
        this.address = address;
    }
}
```

Explanation: Spring provides the `Address` object through the setter method after creating the `Student` object.

Advantages

Constructor Injection: - Ensures required dependencies are provided. - Supports immutable objects. - Makes unit testing easier.

Setter Injection: - Supports optional dependencies. - Allows dependency changes after object creation. - Useful for flexible configuration.

Conclusion

Constructor Injection is preferred for mandatory dependencies, whereas Setter Injection is suitable for optional dependencies that may need to be changed later.

Easy Memory Trick: - Constructor = Dependency through Constructor. - Setter = Dependency through Setter Method.

QUESTION 19

19. Write steps to create a Spring Boot Hello World application using Spring Initializr.

19. Write Steps to Create a Spring Boot Hello World Application Using Spring Initializr.

Spring Initializr is a web-based tool used to create and configure Spring Boot projects easily. It generates the basic project structure with the required dependencies, reducing manual setup.

Steps to Create a Spring Boot Hello World Application

Step 1: Open Spring Initializr

Visit the official website: https://start.spring.io/

Step 2: Configure the Project

Select the following options:

- Project: Maven
- Language: Java
- Spring Boot: Choose a stable version.
- Group: `com.example`
- Artifact: `helloworld`
- Packaging: Jar
- Java: Choose a compatible version.

Step 3: Add Dependencies

Click on **Add Dependencies** and select:

- Spring Web

This dependency is required to develop web applications and REST APIs.

Step 4: Generate the Project

Click the **Generate** button. A ZIP file containing the Spring Boot project will be downloaded.

Step 5: Extract and Import the Project

1. Extract the downloaded ZIP file.
2. Open IntelliJ IDEA, Eclipse or Spring Tool Suite.
3. Import the project as an existing Maven project.

Step 6: Create a Controller

Create a Java class named `HelloController`.

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello World!";
    }
}
```

Step 7: Run the Application

Open the main application class and run it.

```
@SpringBootApplication
public class HelloworldApplication {
    public static void main(String[] args) {
        SpringApplication.run(
            HelloworldApplication.class, args);
    }
}
```

Step 8: Test the Application

Open a web browser and enter:

`http://localhost:8080/hello`

Output:

```
Hello World!
```

Conclusion

Spring Initializr simplifies Spring Boot project creation by automatically generating project files and dependencies. Using Spring Web and a REST Controller, we can easily create and run a Hello World application.

QUESTION 20

20. Apply @ComponentScan to enable Auto Scanning.

20. Apply @ComponentScan to Enable Auto Scanning.

@ComponentScan is an annotation in the Spring Framework used to enable automatic scanning of specified packages. It detects classes annotated with `@Component`, `@Service`, `@Repository` and `@Controller` and registers them as **Spring Beans** in the IoC Container.

It reduces manual bean configuration and simplifies application development.

Steps to Apply @ComponentScan

Step 1: Create a Component Class

```
package com.example.service;

import org.springframework.stereotype.Component;

@Component
public class Student {

    public void display() {
        System.out.println("Hello Student");
    }
}
```

Here, `@Component` marks the Student class as a Spring-managed component.

Step 2: Create a Configuration Class

```
package com.example.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.ComponentScan;

@Configuration
@ComponentScan(basePackages = "com.example.service")
public class AppConfig {
}
```

Here: - `@Configuration` identifies the configuration class. - `@ComponentScan` enables automatic component scanning. - `basePackages` specifies the package to scan.

Step 3: Access the Bean

```
public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(
                AppConfig.class);

        Student student = context.getBean(Student.class);

        student.display();
    }
}
```

Working Process

1. Spring reads the configuration class.
2. It scans the specified package.
3. It detects classes containing component annotations.
4. It automatically registers detected classes as Spring Beans.
5. The application retrieves and uses the required beans.

Output

```
Hello Student
```

Advantages

1. Reduces manual bean configuration.
2. Automatically detects Spring components.
3. Simplifies dependency management.
4. Improves code organization and maintainability.

Conclusion: `@ComponentScan` enables Auto Scanning by detecting annotated classes and registering them as Spring Beans in the IoC Container.

QUESTION 21

21. What is @SpringBootApplication annotation?

21. What is @SpringBootApplication Annotation?

`@SpringBootApplication` is a special annotation in the **Spring Boot Framework** used to mark the main class of a Spring Boot application. It enables automatic configuration, component scanning and configuration support.

It simplifies application setup by combining three important Spring annotations into a single annotation.

Three Main Annotations

1. **@SpringBootConfiguration:**
2. Indicates that the class provides configuration for the Spring Boot application.
3. It is a specialized form of `@Configuration`.
4. **@EnableAutoConfiguration:**
5. Automatically configures the application based on the dependencies available in the project.
6. Reduces the need for manual configuration.
7. **@ComponentScan:**
8. Automatically scans the current package and its sub-packages.
9. Detects classes annotated with `@Component`, `@Service`, `@Repository` and `@Controller`.
10. Registers them as Spring Beans.

Example

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MyApplication {

    public static void main(String[] args) {

        SpringApplication.run(
            MyApplication.class, args);
    }
}
```

Explanation

- `@SpringBootApplication` enables the three main features.
- `SpringApplication.run()` starts the Spring Boot application.
- The application automatically scans components and configures required beans.

Advantages

1. Reduces configuration code.
2. Enables automatic configuration.
3. Supports automatic component scanning.
4. Simplifies Spring Boot application startup.
5. Makes application development easier.

Conclusion

`@SpringBootApplication` is the main annotation used to start a Spring Boot application. It combines `@SpringBootConfiguration`, `@EnableAutoConfiguration` and `@ComponentScan` into one annotation.

Important Exam Point: Remember the three annotations: Configuration + Auto Configuration + Component Scanning.

QUESTION 22

22. Describe how Auto Wiring works internally in Spring

22. Describe How Autowiring Works Internally in Spring.

Autowiring in Spring is a feature of Dependency Injection (DI) that automatically provides the required dependencies to a class without manually creating objects. Spring uses the `@Autowired` annotation to identify where dependencies need to be injected.

Internal Working of Autowiring

Autowiring works internally through the following steps:

1. **Bean Creation:** Spring IoC Container starts creating objects (Spring Beans) based on configuration and component scanning.
2. **Dependency Detection:** Spring detects the `@Autowired` annotation in constructors, fields or setter methods.
3. **Dependency Search:** Spring searches the IoC Container for a suitable bean that matches the required dependency type.
4. **Bean Matching:** Spring checks the available beans to find a matching dependency. If multiple beans of the same type exist, it can use `@Qualifier` to identify the correct bean.
5. **Dependency Injection:** Spring injects the matching bean into the required class.
6. **Bean Initialization:** After dependency injection, Spring completes bean initialization and makes the object available for application use.

Example

```
@Component
class Student {
    void display() {
        System.out.println("Student Details");
    }
}

@Component
class College {

    @Autowired
    private Student student;

    void show() {
        student.display();
    }
}
```

Explanation

- `@Component` registers both classes as Spring Beans.
- `@Autowired` tells Spring to inject the Student dependency into College.
- Spring searches the IoC Container for a matching Student bean.
- It injects the dependency automatically.
- The `show()` method uses the injected object.

Important Points

1. Autowiring reduces manual object creation.
2. It supports constructor, setter and field injection.
3. It helps achieve loose coupling.

4. If multiple matching beans exist, `@Qualifier` can specify the required bean.
5. If no matching bean is found for a required dependency, Spring generally throws an error.

Conclusion

Autowiring internally works by detecting dependencies, searching for matching Spring Beans and injecting them into the required classes through the IoC Container.

QUESTION 23

23. Create a test class to verify Auto Wiring functionality and Describe how Auto Wiring works in Spring Boot.

23. Create a Test Class to Verify Autowiring Functionality and Describe How Autowiring Works in Spring Boot.

Autowiring in Spring Boot is a feature of Dependency Injection (DI) that automatically provides the required objects to a class. It uses the `@Autowired` annotation to inject dependencies without manually creating objects.

1. How Autowiring Works in Spring Boot

1. **Bean Creation:** Spring Boot creates objects (Spring Beans) using component scanning and configuration.
2. **Dependency Detection:** Spring detects the `@Autowired` annotation in a class.
3. **Bean Searching:** It searches the IoC Container for a matching bean.
4. **Dependency Injection:** The matching bean is automatically injected into the required class.
5. **Bean Initialization:** Spring completes initialization and makes the object available for use.

2. Example Program

Step 1: Create Student Class

```
@Component
public class Student {
    public String getName() {
        return "Rahul";
    }
}
```

Step 2: Create College Class

```
@Component
public class College {

    @Autowired
    private Student student;

    public String getStudentName() {
        return student.getName();
    }
}
```

Step 3: Create Test Class

```
@SpringBootTest
class CollegeTest {

    @Autowired
    private College college;

    @Test
    void testAutowired() {
        assertEquals(
            "Rahul",
            college.getStudentName()
        );
    }
}
```

3. Explanation

- `@Component` registers Student and College as Spring Beans.
- `@Autowired` injects the Student object into College.
- `@SpringBootTest` loads the Spring Boot application context for testing.
- `@Test` identifies the test method.
- `assertEquals()` checks whether the actual result matches the expected result.

If the student name is `"Rahul"`, the test passes successfully.

Conclusion

Autowiring in Spring Boot automatically detects and injects required dependencies through the IoC Container. The test class verifies that dependency injection works correctly.

Important: `@Autowired` = Automatic Dependency Injection.

QUESTION 24

24. Explain different bean scopes in Spring and describe the use of loggers with suitable examples.

24. Explain Different Bean Scopes in Spring and Describe the Use of Loggers with Suitable Examples.

1. Bean Scopes in Spring

Bean Scope defines the lifecycle and visibility of a Spring Bean. It determines how many instances of a bean Spring creates and how long those instances remain available.

Spring supports the following bean scopes:

1. **Singleton:** Only one instance of a bean is created per Spring IoC Container. It is the default scope.
2. **Prototype:** A new bean instance is created every time the bean is requested from the container.
3. **Request:** A new bean instance is created for each HTTP request. It is used in web applications.
4. **Session:** One bean instance is created for each HTTP session and remains available throughout that session.
5. **Application:** One bean instance is created per ServletContext and shared across the web application.
6. **WebSocket:** A separate bean instance is created for each WebSocket session.

Example:

```
@Component
@Scope("prototype")
public class Student {
}
```

Here, Spring creates a new Student bean whenever it is requested from the container.

2. Loggers in Spring

Logger is a tool used to record application activities, events, warnings and errors during program execution.

Spring Boot commonly uses **SLF4J** as a logging API with Logback as its default logging implementation.

Uses of Loggers

1. **Debugging:** Helps identify errors in the application.
2. **Monitoring:** Tracks application activities.
3. **Error Tracking:** Records exceptions and failures.
4. **Information Recording:** Displays important application events.

Logger Example

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@RestController
public class StudentController {

    private static final Logger logger =
        LoggerFactory.getLogger(StudentController.class);

    @GetMapping("/student")
    public String getStudent() {

        logger.info("Student API called");

        return "Student Details";
    }
}
```

Explanation: - `LoggerFactory` creates a logger. - `logger.info()` records an informational message. - The message appears in the application logs when the API is called.

Conclusion

Bean scopes control the lifecycle and number of Spring Bean instances, while loggers help monitor application activities and identify errors during execution.

QUESTION 25

25. Describe the role of Logback in Spring Boot.

25. Describe the Role of Logback in Spring Boot.

Logback is a logging framework used in Spring Boot to record application activities, events, warnings and errors during program execution. It is the default logging implementation used by Spring Boot when the standard logging starter is included.

Logback helps developers monitor applications, identify errors and understand application behaviour.

Main Roles of Logback in Spring Boot

1. **Application Monitoring:** Records important application activities, such as application startup and API requests.
2. **Error Tracking:** Records errors and exceptions to help developers identify and fix problems.
3. **Different Logging Levels:** Supports different logging levels to classify messages according to their importance.
4. **File Logging:** Stores log messages in files for future reference and analysis.
5. **Log Formatting:** Controls how log messages are displayed using patterns.
6. **Configuration:** Allows developers to configure logging through files such as `logback-spring.xml`.

Logging Levels in Logback

Level	Purpose
TRACE	Records very detailed information.
DEBUG	Provides debugging information.
INFO	Records normal application activities.
WARN	Indicates potential problems.
ERROR	Records serious errors.

Example

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@RestController
public class StudentController {

    private static final Logger logger =
        LoggerFactory.getLogger(StudentController.class);

    @GetMapping("/student")
    public String getStudent() {

        logger.info("Student API called");
        logger.warn("Warning message");
        logger.error("Error occurred");

        return "Student Details";
    }
}
```

Explanation

- `LoggerFactory` creates a logger for the class.
- `logger.info()` records informational messages.
- `logger.warn()` records warning messages.
- `logger.error()` records error messages.
- Logback processes and displays these messages according to the logging configuration.

Advantages of Logback

1. Helps identify application errors.
2. Supports multiple logging levels.
3. Allows logging into files.
4. Helps monitor application performance and activities.

Conclusion

Logback plays an important role in Spring Boot by providing efficient logging, error tracking and application monitoring. It helps developers maintain and troubleshoot applications easily.

QUESTION 26

26. Explain the concept of AOP and its benefits.

26. Explain the Concept of AOP and Its Benefits.

AOP (Aspect-Oriented Programming) is a programming approach used in the Spring Framework to separate common tasks from the main business logic of an application.

These common tasks are called **Cross-Cutting Concerns**, such as logging, security, transaction management and exception handling.

AOP helps developers manage these tasks separately without repeatedly writing the same code in different classes.

Important Terminologies of AOP

1. **Aspect:** A class that contains common functionality, such as logging or security.
2. **Join Point:** A point during program execution where an aspect can be applied, such as a method execution.
3. **Advice:** The action performed by an aspect at a particular join point.
4. **Pointcut:** An expression that identifies where advice should be executed.
5. **Weaving:** The process of connecting aspects with the application code.

Types of Advice

1. **Before Advice:** Executes before a method.
2. **After Advice:** Executes after a method completes.
3. **After Returning:** Executes when a method completes successfully.
4. **After Throwing:** Executes when a method throws an exception.
5. **Around Advice:** Executes before and after a method.

Example

```
@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* com.example.service.*.*(..))")
    public void logBefore() {
        System.out.println("Method is executing");
    }
}
```

Explanation: - `@Aspect` defines the class as an aspect. - `@Component` registers it as a Spring Bean. - `@Before` executes advice before the matching method. - `execution()` defines which methods should be intercepted.

Benefits of AOP

1. **Code Reusability:** Common functionality can be reused.
2. **Loose Coupling:** Separates business logic from common tasks.
3. **Easy Maintenance:** Changes to common functionality can be made centrally.
4. **Better Readability:** Keeps business logic clean and simple.
5. **Reduced Code Duplication:** Avoids repeating logging and security code.

Conclusion

AOP separates cross-cutting concerns from business logic, making Spring applications more modular, reusable and maintainable.

QUESTION 27

27. Develop a Spring Boot application with Employee & EmployeeService using @Autowired.

27. Develop a Spring Boot Application with Employee & EmployeeService Using @Autowired.

Spring Boot supports Dependency Injection using the `@Autowired` annotation. It automatically injects the required Spring Bean into another class without manually creating objects.

In this application, we create an `Employee` class and an `EmployeeService` class. The `EmployeeService` uses `@Autowired` to access the Employee object.

Step 1: Create Employee Class

```
import org.springframework.stereotype.Component;

@Component
public class Employee {

    public String getEmployeeDetails() {
        return "Employee ID: 101, Name: Rahul";
    }
}
```

Explanation: - `@Component` registers the Employee class as a Spring Bean. - `getEmployeeDetails()` returns employee information.

Step 2: Create EmployeeService Class

```
import org.springframework.stereotype.Service;
import org.springframework.beans.factory.annotation.Autowired;

@Service
public class EmployeeService {

    @Autowired
    private Employee employee;

    public String showEmployee() {
        return employee.getEmployeeDetails();
    }
}
```

Explanation: - `@Service` marks the class as a service component. - `@Autowired` automatically injects the Employee object. - `showEmployee()` calls the Employee method.

Step 3: Create Main Spring Boot Class

```
@SpringBootApplication
public class EmployeeApplication {

    public static void main(String[] args) {
        SpringApplication.run(
            EmployeeApplication.class, args);
    }
}
```

Step 4: Create REST Controller

```
@RestController
public class EmployeeController {

    @Autowired
    private EmployeeService service;

    @GetMapping("/employee")
    public String getEmployee() {
        return service.showEmployee();
    }
}
```

Step 5: Run the Application

Start the Spring Boot application and open the browser:

`http://localhost:8080/employee`

Output:

```
Employee ID: 101, Name: Rahul
```

Working Process

1. Spring creates the Employee Bean.
2. Spring creates the EmployeeService Bean.
3. `@Autowired` injects Employee into EmployeeService.
4. The Controller calls EmployeeService.
5. Employee details are returned to the client.

Conclusion

This application demonstrates how `@Autowired` performs automatic Dependency Injection in Spring Boot, reducing manual object creation and promoting loose coupling.

QUESTION 28

28. What are the major limitations of the JDBC API? Why is Spring Data JPA preferred over traditional JDBC?

28. What Are the Major Limitations of the JDBC API? Why Is Spring Data JPA Preferred Over Traditional JDBC?

JDBC (Java Database Connectivity) is a Java API used to connect Java applications with databases and perform operations such as INSERT, UPDATE, DELETE and SELECT.

However, traditional JDBC requires developers to write more code and manage database operations manually. Spring Data JPA provides a simpler approach using ORM and repository-based operations.

1. Major Limitations of JDBC API

1. **Boilerplate Code:** Developers must write lengthy code for database connections, statements and result processing.
2. **Manual Resource Management:** Connections, statements and result sets must be closed manually to avoid resource leaks.
3. **SQL Dependency:** Developers need to write SQL queries manually for database operations.
4. **Exception Handling:** JDBC requires handling `SQLException` explicitly.
5. **Object Mapping:** Developers must manually convert database records into Java objects.
6. **Difficult Maintenance:** Large applications become difficult to maintain because of repeated database code.

2. Why Spring Data JPA Is Preferred Over JDBC

Spring Data JPA simplifies database operations by providing repository interfaces and automatic query generation.

Example:

```
public interface EmployeeRepository
        extends JpaRepository<Employee, Integer> {

    List<Employee> findByName(String name);
}
```

Here, Spring Data JPA automatically generates the required query from the method name.

3. Difference Between JDBC and Spring Data JPA

JDBC	Spring Data JPA
Requires manual SQL queries.	Supports automatic query generation.
Requires more boilerplate code.	Reduces boilerplate code.
Manual object mapping.	Supports automatic ORM mapping.
Manual resource management.	Provides simplified resource management.
Requires explicit SQL exception handling.	Uses Spring's exception translation.

4. Advantages of Spring Data JPA

1. Reduces database-related coding.
2. Simplifies CRUD operations.
3. Supports pagination and sorting.
4. Improves code reusability.
5. Makes database application development easier.

Conclusion

Spring Data JPA is preferred over traditional JDBC because it reduces coding complexity, provides automatic object mapping and simplifies database operations.

QUESTION 29

29. Compare derived query methods, @Query, and named queries in Spring Data JPA. Give one example of each.

29. Compare Derived Query Methods, @Query, and Named Queries in Spring Data JPA. Give One Example of Each.

Spring Data JPA provides different methods to retrieve data from a database. Three commonly used approaches are **Derived Query Methods, @Query and Named Queries**. They differ in how queries are defined and executed.

1. Derived Query Methods

Derived Query Methods are created by writing method names according to predefined naming conventions. Spring Data JPA automatically generates the required query from the method name.

Example:

```
public interface EmployeeRepository
        extends JpaRepository<Employee, Integer> {

    List<Employee> findByName(String name);
}
```

Explanation: Spring automatically generates a query to find employees whose name matches the given value.

2. @Query Annotation

The `@Query` annotation is used to define custom JPQL or native SQL queries directly inside a repository interface. It is useful for complex database operations.

Example:

```
public interface EmployeeRepository
        extends JpaRepository<Employee, Integer> {

    @Query("SELECT e FROM Employee e WHERE e.salary > :salary")
    List<Employee> findHighSalary(
        @Param("salary") double salary);
}
```

Explanation: This query retrieves employees whose salary is greater than the given amount.

3. Named Queries

Named Queries are predefined queries identified by a unique name. They are commonly declared using `@NamedQuery` on an entity class and referenced through a repository method.

Example:

```
@Entity
@NamedQuery(
    name = "Employee.findAllEmployees",
    query = "SELECT e FROM Employee e"
)
public class Employee {
    @Id
    private int id;
}
```

Repository method:

```
List<Employee> findAllEmployees();
```

Explanation: The repository method uses the predefined named query to retrieve all employees.

4. Comparison

Derived Query	@Query	Named Query
Based on method names.	Uses custom query definitions.	Uses predefined named queries.
Simple queries.	Simple or complex queries.	Reusable predefined queries.
Written in repository methods.	Written directly in repository interfaces.	Usually defined on entity classes or XML.
Automatically generated.	Explicitly written by developer.	Defined separately and referenced by name.

Conclusion

Derived Query Methods are suitable for simple queries, `@Query` is useful for custom queries, and Named Queries are useful for predefined and reusable queries.

QUESTION 30

30. Explain why transactions are required in Spring applications. Demonstrate the use of @Transactional for an update operation.

30. Explain Why Transactions Are Required in Spring Applications. Demonstrate the Use of @Transactional for an Update Operation.

Transactions in Spring applications are used to ensure that database operations are performed safely and consistently. A transaction is a group of database operations treated as a single unit of work.

Spring provides the `@Transactional` annotation to manage database transactions automatically.

1. Why Transactions Are Required

1. **Data Consistency:** Ensures that database data remains correct after operations.
2. **Atomicity:** Either all operations are completed successfully or none of them are applied.
3. **Rollback Support:** If an error occurs, changes can be rolled back to maintain database consistency.
4. **Error Handling:** Helps manage database failures during multiple operations.
5. **Reliable Updates:** Prevents incomplete database updates when an operation fails.

2. ACID Properties

Transactions follow four important ACID properties:

- **Atomicity:** All operations succeed or fail together.
- **Consistency:** Database remains in a valid state.
- **Isolation:** Concurrent transactions do not interfere improperly.
- **Durability:** Committed changes are permanently saved.

3. Example Using @Transactional

Suppose we want to update an employee's salary.

Employee Entity:

```
@Entity
public class Employee {
    @Id
    private int id;
    private double salary;
}
```

Repository:

```
public interface EmployeeRepository
        extends JpaRepository<Employee, Integer> {
}
```

Service Class:

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repository;

    @Transactional
    public void updateSalary(int id, double salary) {

        Employee emp = repository.findById(id)
            .orElseThrow(() ->
                new RuntimeException("Employee not found"));

        emp.setSalary(salary);

        repository.save(emp);
    }
}
```

4. Explanation

1. `@Transactional` starts a transaction when the method is called.
2. The employee is retrieved using `findById()`.
3. The salary is updated using `setSalary()`.
4. `save()` stores the updated employee.
5. If the operation completes successfully, the transaction is committed.
6. If an unchecked exception occurs, the transaction is rolled back by default.

Conclusion

Transactions are required to maintain database consistency and reliability. The `@Transactional` annotation simplifies transaction management by automatically handling commit and rollback operations.

QUESTION 31

31. Implement pagination and sorting for the User entity using Pageable and Sort. Explain how the page number, page size, and sorting field are specified.

31. Implement Pagination and Sorting for the User Entity Using Pageable and Sort. Explain How the Page Number, Page Size, and Sorting Field Are Specified.

Pagination in Spring Data JPA is a technique used to divide a large number of database records into smaller pages. **Sorting** is used to arrange records in ascending or descending order based on a particular field.

Spring Data JPA provides `Pageable`, `PageRequest` and `Sort` to implement pagination and sorting easily.

1. Repository Interface

```
public interface UserRepository
        extends JpaRepository<User, Integer> {
}
```

Here, `JpaRepository` provides built-in methods for pagination and sorting.

2. Implement Pagination and Sorting

```
@Service
public class UserService {

    @Autowired
    private UserRepository repository;

    public Page<User> getUsers() {

        Pageable pageable =
            PageRequest.of(0, 5,
                Sort.by("name").ascending());

        return repository.findAll(pageable);
    }
}
```

3. Explanation

The `PageRequest.of()` method specifies three important parameters:

1. **Page Number:**
2. Specifies which page should be retrieved.
3. Page numbering starts from `0`.
4. Example: `0` represents the first page.
5. **Page Size:**
6. Specifies the number of records displayed on each page.
7. Example: `5` means five user records per page.
8. **Sorting Field:**
9. Specifies the field used to arrange records.
10. Example: `Sort.by("name").ascending()` sorts users by name in ascending order.
11. `descending()` sorts records in descending order.

4. Example

```
Pageable pageable = PageRequest.of(
    1, 10, Sort.by("id").descending()
);

Page<User> users = repository.findAll(pageable);
```

Output: - Page Number: 1 (Second Page) - Page Size: 10 records - Sorting Field: `id` - Sorting Order: Descending

5. Advantages

1. Reduces unnecessary data retrieval.
2. Improves application performance.
3. Makes large datasets easier to manage.
4. Supports flexible sorting.

Conclusion

Spring Data JPA uses `Pageable`, `PageRequest` and `Sort` to implement pagination and sorting efficiently. It allows developers to specify the page number, page size and sorting field without writing manual SQL queries.

QUESTION 32

32. What are Web Services? Explain why Web Services are used in distributed applications.

32. What Are Web Services? Explain Why Web Services Are Used in Distributed Applications.

Web Services are software components that allow different applications to communicate and exchange data over a network using standard web protocols such as HTTP. They enable applications developed using different programming languages and operating systems to interact with each other.

Web Services are commonly used in **distributed applications**, where different components run on separate computers or servers but work together to complete a task.

1. Why Web Services Are Used in Distributed Applications

1. **Communication Between Applications:** Web Services allow applications running on different systems to communicate and exchange information.
2. **Platform Independence:** They support communication between applications developed using different programming languages and operating systems.
3. **Interoperability:** They allow different software systems to work together without depending on their internal implementation.
4. **Reusability:** The same Web Service can be accessed by multiple applications, reducing repeated development.
5. **Easy Integration:** They help connect different systems, databases and applications over a network.
6. **Scalability:** Services can be deployed and maintained separately, making it easier to expand distributed applications.

2. Working of Web Services

The basic communication process involves three steps:

1. **Service Provider:** Creates and provides the Web Service.
2. **Service Requester:** Sends a request to access the service.

3. **Service Response:** The provider processes the request and returns the required data.

3. Simple Diagram

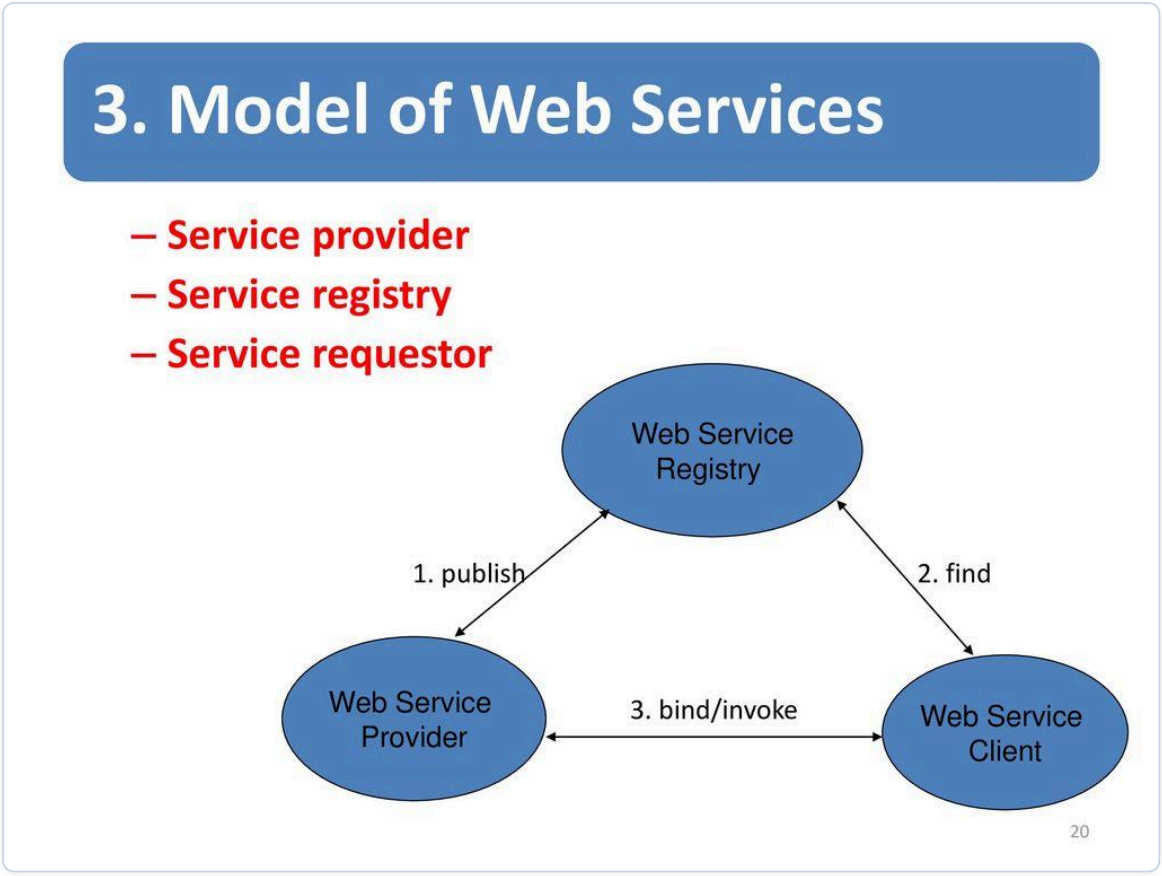


Figure 32.1: Distributed Web Services Architecture (Client-Provider Request-Response Model)

4. Example

Consider an online shopping application.

- The shopping application requests payment processing.
- The Payment Web Service receives the request.
- It processes the payment.
- It sends the payment status back to the shopping application.

Thus, two different applications communicate through a Web Service.

5. Common Types of Web Services

1. **SOAP:** Uses XML-based messaging for communication.
2. **REST:** Uses HTTP methods such as GET, POST, PUT and DELETE.

Conclusion

Web Services are important in distributed applications because they provide platform-independent communication, interoperability, reusability and easy integration between different software systems.

QUESTION 33

1. Create a REST API to retrieve User information using both `@PathVariable` and `@RequestParam`. Test the API using Postman.

1. Create a REST API to Retrieve User Information Using Both `@PathVariable` and `@RequestParam`. Test the API Using Postman.

REST API is used to allow applications to communicate with each other through HTTP requests. In Spring Boot, `@PathVariable` and `@RequestParam` are used to receive values from client requests and retrieve specific user information.

1. Create User Class

```
public class User {
    private int id;
    private String name;
    private String email;

    public User(int id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
}
```

2. Create REST Controller

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    List<User> users = Arrays.asList(
        new User(1, "Rahul", "rahul@gmail.com"),
        new User(2, "Amit", "amit@gmail.com")
    );

    // Using PathVariable
    @GetMapping("/{id}")
    public User getUserById(@PathVariable int id) {
        return users.stream()
            .filter(u -> u.getId() == id)
            .findFirst()
            .orElse(null);
    }

    // Using RequestParam
    @GetMapping("/search")
    public User getUserByName(
            @RequestParam String name) {
        return users.stream()
            .filter(u -> u.getName().equalsIgnoreCase(name))
            .findFirst()
            .orElse(null);
    }
}
```

3. Explanation

@PathVariable: - Extracts values directly from the URL path. - Example: `/api/users/1` - Retrieves the user whose ID is 1.

@RequestParam: - Extracts values from URL query parameters. - Example: `/api/users/search?name=Rahul` - Retrieves the user whose name is Rahul.

4. Test Using Postman

Test 1: Using @PathVariable

- Method: GET
- URL: `http://localhost:8080/api/users/1`

Expected JSON Output:

```
{
  "id": 1,
  "name": "Rahul",
  "email": "rahul@gmail.com"
}
```

Test 2: Using @RequestParam

- Method: GET
- URL: `http://localhost:8080/api/users/search?name=Rahul`

Expected JSON Output:

```
{
  "id": 1,
  "name": "Rahul",
  "email": "rahul@gmail.com"
}
```

Conclusion

`@PathVariable` retrieves values from URL paths, whereas `@RequestParam` retrieves values from query parameters. Both annotations help create flexible REST APIs in Spring Boot and can be tested using Postman.

QUESTION 34

34. Explain the different types of Web Services. Compare SOAP and REST.

34. Explain the Different Types of Web Services. Compare SOAP and REST.

Web Services are software components that allow different applications to communicate and exchange data over a network using standard protocols. They help applications developed using different programming languages and platforms interact with each other.

1. Types of Web Services

There are two major types of Web Services:

1. SOAP (Simple Object Access Protocol)

- SOAP is a protocol used for exchanging structured information between applications.
- It uses **XML (Extensible Markup Language)** for message formatting.
- It commonly works over HTTP and supports other transport protocols.
- It provides standards for security and reliable messaging.
- Example: Banking and enterprise applications.

2. REST (Representational State Transfer)

- REST is an architectural style used to develop Web Services.

- It uses HTTP methods such as GET, POST, PUT and DELETE.
- It supports data formats such as JSON and XML.
- It is lightweight and easy to implement.
- Example: E-commerce applications and mobile applications.

2. Difference Between SOAP and REST

SOAP	REST
1. SOAP is a communication protocol.	REST is an architectural style.
2. Uses XML for messages.	Supports JSON, XML and other formats.
3. Generally requires more message overhead.	Usually has less message overhead.
4. Supports standards such as WS-Security.	Commonly uses HTTPS and token-based security.
5. Can work over HTTP, SMTP and other transports.	Primarily uses HTTP/HTTPS.
6. Suitable for enterprise and financial systems.	Suitable for web and mobile applications.
7. Uses SOAP envelopes for messages.	Uses HTTP methods to access resources.

3. Examples

SOAP Request:

```
<soap:Envelope>
    <soap:Body>
        <getStudent>
            <id>101</id>
        </getStudent>
    </soap:Body>
</soap:Envelope>
```

REST Request:

```
GET /api/students/101
```

4. Applications

- **SOAP:** Banking systems, payment processing and enterprise applications.
- **REST:** Social media applications, e-commerce websites and mobile applications.

Conclusion

SOAP and REST are two major approaches to building Web Services. SOAP provides a standardized messaging protocol, whereas REST provides a lightweight architectural approach for communication between applications.

QUESTION 35

35. Explain Service-Oriented Architecture (SOA) and its main principles.

35. Explain Service-Oriented Architecture (SOA) and Its Main Principles.

Service-Oriented Architecture (SOA) is a software design approach in which an application is divided into independent services. Each service performs a specific business function and communicates with other services over a network.

SOA allows different applications to share services and exchange information without depending on their internal implementation.

1. Main Components of SOA

1. **Service Provider:** Creates and provides services to other applications.
2. **Service Consumer:** Requests and uses the services provided by another application.
3. **Service Registry:** Stores information about available services so consumers can discover them.

2. Main Principles of SOA

1. Loose Coupling - Services have minimal dependency on each other. - Changes in one service have less impact on other services.

2. Reusability - A service can be reused by multiple applications. - It reduces duplicate development work.

3. Standardized Service Contract - Every service provides a defined interface or contract. - Consumers know how to communicate with the service.

4. Service Autonomy - Each service manages its own functionality and resources. - Services can operate independently.

5. Service Discoverability - Services can be identified and accessed through service descriptions or registries.

6. Interoperability - Services developed using different programming languages and platforms can communicate with each other.

3. SOA Architecture Diagram

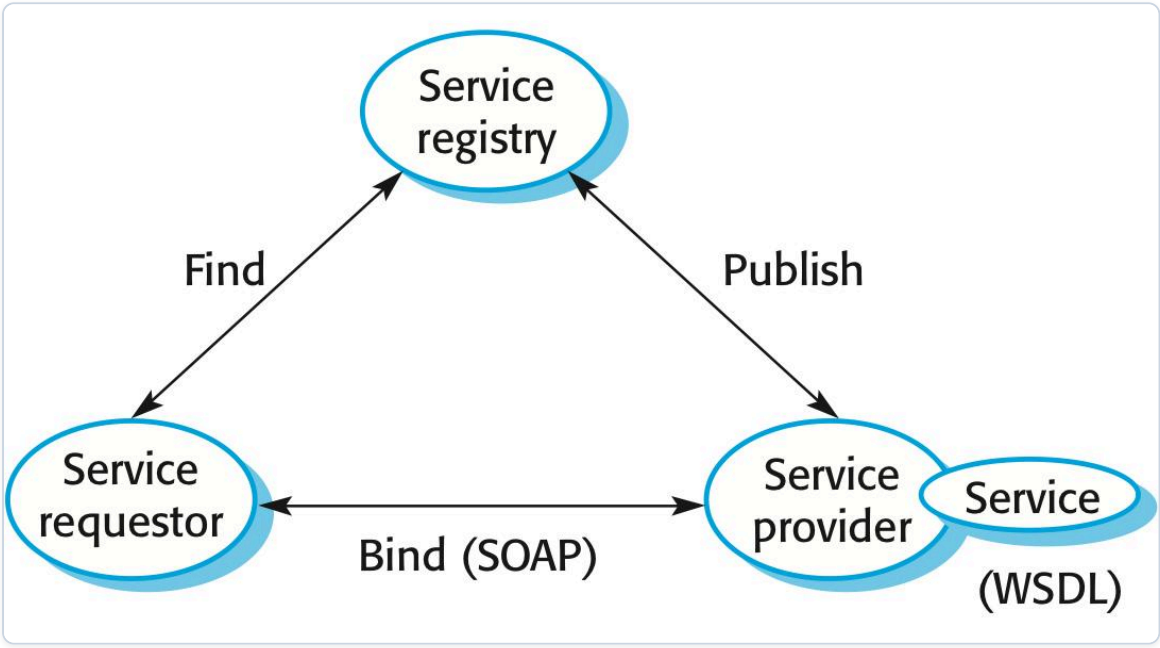


Figure 35.1: Service-Oriented Architecture (SOA) Publish-Find-Bind Model

4. Example

Consider an online shopping application.

- **Payment Service:** Processes customer payments.
- **Inventory Service:** Checks product availability.
- **Delivery Service:** Manages product delivery.
- **Customer Service:** Manages customer information.

These independent services communicate with each other to complete an online purchase.

5. Advantages of SOA

1. Improves service reusability.
2. Reduces application complexity.

3. Supports integration of different systems.
4. Makes applications flexible and scalable.
5. Simplifies maintenance.

Conclusion

SOA organizes applications into independent and reusable services. Its main principles include loose coupling, reusability, service autonomy, standardized contracts and interoperability.

QUESTION 36

36. Why is REST widely used for modern Web APIs? Explain with suitable examples.

36. Why Is REST Widely Used for Modern Web APIs? Explain with Suitable Examples.

REST (Representational State Transfer) is an architectural style used to develop Web APIs that allow different applications to communicate and exchange data over HTTP. REST is widely used because it is lightweight, simple, flexible and easy to implement.

It allows applications developed using different programming languages and platforms to communicate with each other.

1. Reasons Why REST Is Widely Used

1. Lightweight Communication - REST uses simple HTTP requests and responses. - It commonly uses JSON, which is easy to read and process.

2. Platform Independence - REST APIs work with different programming languages and operating systems. - Example: A Java backend can communicate with an Android application.

3. Easy Implementation - REST uses standard HTTP methods such as GET, POST, PUT and DELETE. - Developers can easily create and access REST APIs.

4. Scalability - REST supports stateless communication, making it easier to handle multiple client requests.

5. Flexibility - REST supports different data formats, including JSON and XML. - It can be used in web applications, mobile applications and cloud systems.

6. Caching Support - REST allows responses to be cached when appropriate, improving performance and reducing repeated server requests.

2. Common HTTP Methods in REST

Method	Purpose	Example
GET	Retrieve data	Get student details
POST	Create data	Add a new student
PUT	Update data	Update student details
DELETE	Remove data	Delete a student

3. Suitable Example

Consider a Student Management System.

GET Request:

```
GET /api/students/101
```

JSON Response:

```
{
  "id": 101,
  "name": "Rahul",
  "course": "AI and ML"
}
```

Here, the REST API retrieves student information and returns it in JSON format.

4. Applications of REST APIs

1. E-commerce applications.
2. Social media platforms.
3. Banking applications.
4. Mobile applications.
5. Cloud-based applications.

Conclusion

REST is widely used for modern Web APIs because it provides lightweight communication, platform independence, scalability, flexibility and easy integration between different applications.

QUESTION 37

1. Explain the differences between `@PathVariable`, `@RequestParam`, and `@MatrixVariable` with suitable examples.

1. Explain the Differences Between `@PathVariable`, `@RequestParam`, and `@MatrixVariable` with Suitable Examples.

In **Spring Boot REST APIs**, `@PathVariable`, `@RequestParam` and `@MatrixVariable` are annotations used to extract values from different parts of a URL. They help controllers receive data from client requests.

1. @PathVariable

`@PathVariable` is used to extract values directly from the URL path.

Example:

```
@GetMapping("/users/{id}")
public String getUser(@PathVariable int id) {
    return "User ID: " + id;
}
```

URL:

```
http://localhost:8080/users/101
```

Output:

```
User ID: 101
```

Here, `101` is extracted from the URL path and assigned to the `id` variable.

2. @RequestParam

`@RequestParam` is used to extract values from query parameters in a URL. Query parameters appear after the `?` symbol.

Example:

```
@GetMapping("/users")
public String getUser(
    @RequestParam String name) {
    return "User Name: " + name;
}
```

URL:

```
http://localhost:8080/users?name=Rahul
```

Output:

```
User Name: Rahul
```

Here, `Rahul` is extracted from the query parameter.

3. @MatrixVariable

`@MatrixVariable` is used to extract key-value pairs from a URL path segment. Matrix variables are separated using semicolons (`;`).

Example:

```
@GetMapping("/users/{details}")
public String getUser(
    @MatrixVariable String name) {
    return "User Name: " + name;
}
```

URL:

```
http://localhost:8080/users/details;name=Rahul
```

Output:

```
User Name: Rahul
```

Note: Matrix variables must be enabled in Spring MVC configuration when using the default setup.

4. Difference Between Them

Annotation	Purpose	URL Format
`@PathVariable`	Extracts path values.	`/users/101`
`@RequestParam`	Extracts query parameters.	`/users?name=Rahul`
`@MatrixVariable`	Extracts matrix parameters.	`/users;name=Rahul`

Conclusion

All three annotations extract values from URLs, but their locations are different. `@PathVariable` extracts path values, `@RequestParam` extracts query parameters, and `@MatrixVariable` extracts semicolon-separated matrix parameters.

QUESTION 38

38. Explain Spring REST and describe the steps involved in creating a REST Controller using Spring Boot.

38. Explain Spring REST and Describe the Steps Involved in Creating a REST Controller Using Spring Boot.

Spring REST is a feature of the Spring Framework used to develop RESTful Web Services. It allows different applications to communicate and exchange data over HTTP using methods such as GET, POST, PUT and DELETE.

Spring Boot simplifies REST API development by providing annotations such as `@RestController`, `@RequestMapping`, `@GetMapping` and `@PostMapping`.

1. Steps to Create a REST Controller Using Spring Boot

Step 1: Create a Spring Boot Project

- Open Spring Initializr.
- Select Maven and Java.
- Add the **Spring Web** dependency.
- Generate and import the project into an IDE.

Step 2: Create a Model Class

Create a Java class named `Student`.

```
public class Student {
    private int id;
    private String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int getId() { return id; }
    public String getName() { return name; }
}
```

Step 3: Create a REST Controller

```
@RestController
@RequestMapping("/api")
public class StudentController {

    @GetMapping("/student")
    public Student getStudent() {
        return new Student(101, "Rahul");
    }
}
```

Step 4: Run the Application

Run the main Spring Boot application class using `SpringApplication.run()`.

Step 5: Test the REST API

Open a browser or Postman and enter:

```
http://localhost:8080/api/student
```

Expected JSON Output:

```
{
  "id": 101,
  "name": "Rahul"
}
```

2. Important REST Controller Annotations

1. `@RestController`: Defines a REST controller that returns data directly.
2. `@RequestMapping`: Defines the base URL for API endpoints.
3. `@GetMapping`: Handles GET requests.
4. `@PostMapping`: Handles POST requests.
5. `@PutMapping`: Handles PUT requests.
6. `@DeleteMapping`: Handles DELETE requests.

Conclusion

Spring REST makes it easier to develop RESTful APIs using Spring Boot. A REST Controller handles client requests and returns responses, commonly in JSON format.

QUESTION 39

39. Create a REST API to retrieve User information using both @PathVariable and @RequestParam. Test the API using Postman.

39. Create a REST API to Retrieve User Information Using Both @PathVariable and @RequestParam. Test the API Using Postman.

REST API allows different applications to communicate and exchange data over HTTP. In Spring Boot, `@PathVariable` and `@RequestParam` are used to receive values from client requests and retrieve specific user information.

Step 1: Create User Class

Create a Java class named `User`.

```
public class User {
    private int id;
    private String name;
    private String email;

    public User(int id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
}
```

Step 2: Create REST Controller

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    List<User> users = Arrays.asList(
        new User(1, "Rahul", "rahul@gmail.com"),
        new User(2, "Amit", "amit@gmail.com")
    );

    // Using PathVariable
    @GetMapping("/{id}")
    public User getUserById(@PathVariable int id) {
        return users.stream()
            .filter(u -> u.getId() == id)
            .findFirst()
            .orElse(null);
    }

    // Using RequestParam
    @GetMapping("/search")
    public User getUserByName(
            @RequestParam String name) {
        return users.stream()
            .filter(u -> u.getName()
            .equalsIgnoreCase(name))
            .findFirst()
            .orElse(null);
    }
}
```

Step 3: Understanding the Annotations

1. **@PathVariable:** Extracts values directly from the URL path. Example: `/api/users/1` retrieves the user with ID 1.
2. **@RequestParam:** Extracts values from query parameters. Example: `/api/users/search?name=Rahul` retrieves the user named Rahul.

Step 4: Test Using Postman

Test 1: Using @PathVariable

- Method: GET
- URL: `http://localhost:8080/api/users/1`

Expected Output:

```
{
  "id": 1,
  "name": "Rahul",
  "email": "rahul@gmail.com"
}
```

Test 2: Using @RequestParam

- Method: GET
- URL: `http://localhost:8080/api/users/search?name=Rahul`

Expected Output:

```
{
  "id": 1,
  "name": "Rahul",
  "email": "rahul@gmail.com"
}
```

Conclusion

`@PathVariable` retrieves values from URL paths, whereas `@RequestParam` retrieves values from query parameters. Both annotations help develop flexible REST APIs in Spring Boot and can be tested using Postman.

QUESTION 40

40. Explain data validation, exception handling, CORS, and security in Spring REST applications.

40. Explain Data Validation, Exception Handling, CORS, and Security in Spring REST Applications.

Spring REST Applications use different mechanisms to ensure that APIs are reliable, secure and handle client requests properly. Four important concepts are Data Validation, Exception Handling, CORS and Security.

1. Data Validation

Data Validation is the process of checking whether the input data received from clients is correct and valid before processing it.

Spring Boot uses annotations such as:

- `@NotNull` – Value must not be null.
- `@NotBlank` – String must not be empty or contain only spaces.
- `@Size` – Defines minimum and maximum length.
- `@Email` – Checks whether the input is a valid email address.

Example:

```
public class User {
    @NotBlank
    private String name;

    @Email
    private String email;
}
```

`@Valid` is used in controller methods to trigger validation.

2. Exception Handling

Exception Handling manages errors that occur during API execution and returns meaningful error responses to clients.

Important annotations:

- `@ExceptionHandler` – Handles specific exceptions.
- `@ControllerAdvice` – Provides global exception handling.
- `@RestControllerAdvice` – Handles exceptions globally and returns response data directly.

Example:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public String handleException(Exception ex) {
        return "Something went wrong";
    }
}
```

3. CORS (Cross-Origin Resource Sharing)

CORS is a security mechanism that controls whether a web application from one origin can access resources from another origin.

Spring REST uses `@CrossOrigin` to allow requests from specified origins.

Example:

```
@CrossOrigin(origins = "http://localhost:3000")
@RestController
public class UserController {
}
```

Here, requests from the specified frontend origin are allowed, subject to other server security rules.

4. Security

Security protects REST APIs from unauthorized access and ensures that only permitted users can access resources.

Main methods include:

1. Authentication – Verifies user identity.
2. Authorization – Controls user permissions.
3. HTTPS – Encrypts communication.
4. JWT – Uses tokens for authentication.

Conclusion

Data Validation checks input, Exception Handling manages errors, CORS controls cross-origin access, and Security protects REST APIs from unauthorized access. Together, they make Spring REST applications reliable and secure.

QUESTION 41

41. Create a Spring Boot REST application for managing User resources. Create a UserController and implement CRUD operations. Test the APIs using Postman or curl.

41. Create a Spring Boot REST Application for Managing User Resources. Create a UserController and Implement CRUD Operations. Test the APIs Using Postman or curl.

CRUD (Create, Read, Update, Delete) operations are used to manage resources in a REST API. Spring Boot provides annotations such as `@RestController`, `@PostMapping`, `@GetMapping`, `@PutMapping` and `@DeleteMapping` to implement these operations.

Step 1: Create Spring Boot Project

Open Spring Initializr and select:

- Project: Maven
- Language: Java
- Dependency: Spring Web

Generate and import the project into your IDE.

Step 2: Create User Class

```
public class User {
    private int id;
    private String name;
    private String email;

    public User(int id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

Step 3: Create UserController

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    private List<User> users = new ArrayList<>();

    // CREATE
    @PostMapping
    public User addUser(@RequestBody User user) {
        users.add(user);
        return user;
    }

    // READ
    @GetMapping
    public List<User> getUsers() {
        return users;
    }

    // UPDATE
    @PutMapping("/{id}")
    public User updateUser(
            @PathVariable int id,
            @RequestBody User updatedUser) {

        for (User user : users) {
            if (user.getId() == id) {
                user.setName(updatedUser.getName());
                user.setEmail(updatedUser.getEmail());
                return user;
            }
        }
        return null;
    }

    // DELETE
    @DeleteMapping("/{id}")
    public String deleteUser(@PathVariable int id) {
        users.removeIf(user -> user.getId() == id);
        return "User deleted successfully";
    }
}
```

Step 4: Test APIs Using Postman

Operation	HTTP Method	URL
Create	POST	`/api/users`
Read	GET	`/api/users`
Update	PUT	`/api/users/1`
Delete	DELETE	`/api/users/1`

POST Request Body:

```
{
  "id": 1,
  "name": "Rahul",
  "email": "rahul@gmail.com"
}
```

Expected Output:

```
{
  "id": 1,
  "name": "Rahul",
  "email": "rahul@gmail.com"
}
```

Step 5: Test Using curl

```
curl http://localhost:8080/api/users
```

Conclusion

This application demonstrates all four CRUD operations using Spring Boot REST APIs. `UserController` handles client requests and manages User resources through HTTP methods. This example uses an in-memory list; data is not permanently stored in a database.

QUESTION 42

1. Create a Spring Boot RESTful Web Service for a `User` resource. Implement CRUD operations and test the APIs using Postman or curl.

1. Create a Spring Boot RESTful Web Service for a User Resource. Implement CRUD Operations and Test the APIs Using Postman or curl.

Spring Boot RESTful Web Service is used to create APIs that allow clients to perform operations on resources using HTTP methods. CRUD stands for **Create, Read, Update and Delete**. These operations help manage User information.

Step 1: Create Spring Boot Project

Open Spring Initializr and select:

- Project: Maven
- Language: Java
- Dependency: Spring Web

Generate and import the project into IntelliJ IDEA or Eclipse.

Step 2: Create User Class

```
public class User {
    private int id;
    private String name;
    private String email;

    public User(int id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }

    public void setName(String name) {
        this.name = name;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}
```

Step 3: Create UserController

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    private List<User> users = new ArrayList<>();

    // CREATE
    @PostMapping
    public User addUser(@RequestBody User user) {
        users.add(user);
        return user;
    }

    // READ
    @GetMapping
    public List<User> getUsers() {
        return users;
    }

    // UPDATE
    @PutMapping("/{id}")
    public User updateUser(
        @PathVariable int id,
        @RequestBody User updatedUser) {

        for (User user : users) {
            if (user.getId() == id) {
                user.setName(updatedUser.getName());
                user.setEmail(updatedUser.getEmail());
                return user;
            }
        }
        return null;
    }

    // DELETE
    @DeleteMapping("/{id}")
    public String deleteUser(@PathVariable int id) {
        users.removeIf(user -> user.getId() == id);
        return "User deleted successfully";
    }
}
```

Step 4: Test APIs Using Postman

Operation	Method	URL
Create	POST	`/api/users`
Read	GET	`/api/users`
Update	PUT	`/api/users/1`
Delete	DELETE	`/api/users/1`

POST Request Body:

```
{
  "id": 1,
  "name": "Rahul",
  "email": "rahul@gmail.com"
}
```

Expected JSON Response:

```
{
  "id": 1,
  "name": "Rahul",
  "email": "rahul@gmail.com"
}
```

Step 5: Test Using curl

```
curl http://localhost:8080/api/users
```

Explanation

- `@RestController` handles REST API requests.
- `@RequestMapping` defines the base URL.
- `@PostMapping` creates a user.
- `@GetMapping` retrieves users.
- `@PutMapping` updates user details.
- `@DeleteMapping` deletes a user.
- `@RequestBody` receives JSON data.
- `@PathVariable` extracts the user ID from the URL.

Conclusion

Spring Boot simplifies RESTful Web Service development by providing annotations for CRUD operations. Postman or curl can be used to test the APIs. This example stores users in an in-memory list rather than a permanent database.

QUESTION 43

43. Apply Java-based configuration techniques to set up and manage an IoC container in a Spring application.

43. Apply Java-Based Configuration Techniques to Set Up and Manage an

IoC Container in a Spring Application.

Java-Based Configuration in Spring is a technique used to configure and manage application objects (Spring Beans) using Java classes and annotations instead of XML configuration files.

The **IoC (Inversion of Control) Container** manages the creation, configuration, dependency injection and lifecycle of Spring Beans.

1. Important Annotations

1. `@Configuration`: Indicates that a Java class contains Spring configuration.
2. `@Bean`: Defines a method whose returned object is registered as a Spring Bean.
3. `@ComponentScan`: Automatically scans packages for Spring components.
4. `@Autowired`: Automatically injects required dependencies.

2. Steps to Configure IoC Container

Step 1: Create a Student Class

```
public class Student {
    public void display() {
        System.out.println("Student Details");
    }
}
```

Step 2: Create a Configuration Class

```
@Configuration
public class AppConfig {

    @Bean
    public Student student() {
        return new Student();
    }
}
```

Here, `@Configuration` defines the configuration class, and `@Bean` registers the Student object in the IoC Container.

Step 3: Create the Main Class

```
public class Main {
    public static void main(String[] args) {

        ApplicationContext context =
            new AnnotationConfigApplicationContext(
                AppConfig.class);

        Student s = context.getBean(Student.class);

        s.display();
    }
}
```

3. Working of IoC Container

1. The application loads the Java configuration class.
2. Spring creates the IoC Container.
3. The container reads the `@Bean` annotation.
4. It creates and registers the Student object as a Spring Bean.
5. `getBean()` retrieves the required object.
6. The application uses the retrieved object.

4. Output

```
Student Details
```

5. Advantages

1. Eliminates XML configuration.
2. Simplifies bean management.
3. Supports Dependency Injection.
4. Improves code maintainability.
5. Provides centralized object management.

Conclusion

Java-Based Configuration helps set up and manage the Spring IoC Container using Java classes and annotations. It simplifies bean creation, configuration and dependency management without requiring XML files.

QUESTION 44

44. Compare Constructor and Setter Injection with example

44. Compare Constructor and Setter Injection with Example.

Constructor Injection and **Setter Injection** are two types of Dependency Injection (DI) in the Spring Framework. They are used to provide required dependencies (objects) to a class. Both help achieve loose coupling between application components.

1. Constructor Injection

Constructor Injection is a technique in which dependencies are provided through the class constructor when an object is created.

Example:

```
@Component
class Student {
    private final Address address;

    @Autowired
    public Student(Address address) {
        this.address = address;
    }
}
```

Explanation: - Spring provides the `Address` object through the constructor. - Dependency is injected during object creation. - It is suitable for mandatory dependencies.

2. Setter Injection

Setter Injection is a technique in which dependencies are provided through setter methods after an object is created.

Example:

```
@Component
class Student {
    private Address address;

    @Autowired
    public void setAddress(Address address) {
        this.address = address;
    }
}
```

Explanation: - Spring provides the `Address` object through the setter method. - Dependency is injected after object creation. - It is suitable for optional dependencies.

3. Difference Between Constructor and Setter Injection

Constructor Injection	Setter Injection
1. Uses a constructor to inject dependencies.	Uses setter methods to inject dependencies.
2. Dependencies are provided during object creation.	Dependencies are provided after object creation.
3. Suitable for mandatory dependencies.	Suitable for optional dependencies.
4. Supports immutable fields using `final`.	Dependencies can be changed later.
5. Ensures required dependencies are available during construction.	Allows dependencies to be configured later.
6. Generally preferred for required dependencies.	Useful for optional dependencies.

4. Advantages

Constructor Injection: - Ensures required dependencies are provided. - Supports immutable objects. - Makes testing easier.

Setter Injection: - Supports optional dependencies. - Allows dependency modification. - Provides flexible configuration.

Conclusion

Constructor Injection is preferred for mandatory dependencies, whereas Setter Injection is suitable for optional dependencies. Both are used by Spring to achieve Dependency Injection and reduce tight coupling between classes.

QUESTION 45

45. Discuss about Dependency Injection?

45. Discuss About Dependency Injection (DI).

Dependency Injection (DI) is a design pattern in the Spring Framework in which the required objects (dependencies) are provided to a class by the Spring IoC Container instead of the class creating those objects manually.

It is a method of implementing **Inversion of Control (IoC)**. DI helps develop loosely coupled, reusable and maintainable applications.

1. Types of Dependency Injection

Spring supports three main types of Dependency Injection:

1. Constructor Injection - Dependencies are provided through the class constructor. - It is suitable for mandatory dependencies.

Example:

```
@Component
class Student {
    private final Address address;

    @Autowired
    Student(Address address) {
        this.address = address;
    }
}
```

2. Setter Injection - Dependencies are provided using setter methods. - It is suitable for optional dependencies.

Example:

```
@Component
class Student {
    private Address address;

    @Autowired
    public void setAddress(Address address) {
        this.address = address;
    }
}
```

3. Field Injection - Dependencies are directly injected into class variables. - It uses the `@Autowired` annotation.

Example:

```
@Component
class Student {
    @Autowired
    private Address address;
}
```

2. How Dependency Injection Works

1. Spring creates the required objects as Spring Beans.
2. It identifies the dependencies between classes.
3. The IoC Container searches for the required beans.
4. It automatically injects the matching objects.
5. The application uses the injected dependencies.

3. Advantages of Dependency Injection

1. **Loose Coupling:** Reduces dependency between classes.
2. **Code Reusability:** Allows objects to be reused.
3. **Easy Testing:** Makes unit testing easier.
4. **Easy Maintenance:** Makes applications easier to modify.
5. **Reduced Code:** Avoids manual object creation.

Conclusion

Dependency Injection simplifies object management by allowing Spring to provide required dependencies automatically. It is an important feature of the Spring Framework that helps create flexible and maintainable applications.

QUESTION 46

46. Explain about Auto Scanning in brief

46. Explain About Auto Scanning in Brief.

Auto Scanning in the Spring Framework is a feature that automatically detects classes annotated with Spring annotations and registers them as **Spring Beans** in the IoC Container.

It uses the `@ComponentScan` annotation to scan specified packages and identify components. It reduces manual bean configuration and simplifies application development.

1. Annotations Used in Auto Scanning

1. **@Component:** Marks a Java class as a Spring-managed component.
2. **@Service:** Identifies a class containing business logic.
3. **@Repository:** Identifies a class responsible for database operations.
4. **@Controller:** Identifies a class that handles web requests.
5. **@RestController:** Identifies a class that handles REST API requests.
6. **@ComponentScan:** Specifies which packages Spring should scan.

2. Example

Student Class:

```
@Component
public class Student {
    public void display() {
        System.out.println("Hello Student");
    }
}
```

Configuration Class:

```
@Configuration
@ComponentScan(basePackages = "com.example")
public class AppConfig {
}
```

3. How Auto Scanning Works

1. Developers add annotations such as `@Component` to Java classes.
2. `@ComponentScan` specifies the package to scan.
3. Spring scans the specified package.
4. It detects classes containing component annotations.
5. Spring automatically registers detected classes as Beans.
6. These Beans can be used through Dependency Injection.

4. Advantages of Auto Scanning

1. Reduces manual bean configuration.
2. Simplifies application development.
3. Supports automatic dependency management.
4. Improves code organization.
5. Makes applications easier to maintain.

Conclusion

Auto Scanning automatically detects and registers Spring components as Beans using annotations and `@ComponentScan`. It reduces manual configuration and simplifies object management in Spring applications.

QUESTION 47

47. Discuss about Creating a Spring Boot Application.

47. Discuss About Creating a Spring Boot Application.

Spring Boot is an open-source Java framework used to develop standalone, production-ready applications quickly. It simplifies Spring application development by providing auto-configuration, starter dependencies and an embedded server.

Steps to Create a Spring Boot Application

Step 1: Open Spring Initializr

Visit the official website: https://start.spring.io/

Step 2: Configure the Project

Select the following options:

- Project: Maven
- Language: Java
- Group: `com.example`
- Artifact: `helloworld`
- Packaging: Jar
- Java: Compatible version

Step 3: Add Dependencies

Click on Add Dependencies and select:

- Spring Web

This dependency is required to develop REST APIs and web applications.

Step 4: Generate the Project

Click the **Generate** button to download the project as a ZIP file.

Step 5: Import the Project

1. Extract the downloaded ZIP file.
2. Open IntelliJ IDEA, Eclipse or Spring Tool Suite.
3. Import the project as a Maven project.

Step 6: Create a REST Controller

Create a class named `HelloController`.

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello World!";
    }
}
```

Step 7: Run the Application

The main application class is:

```
@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(
            DemoApplication.class, args);
    }
}
```

Run this class to start the Spring Boot application.

Step 8: Test the Application

Open a browser or Postman and enter:

```
http://localhost:8080/hello
```

Output:

```
Hello World!
```

Advantages of Spring Boot

1. Reduces manual configuration.
2. Provides automatic configuration.
3. Supports embedded servers like Tomcat.
4. Simplifies application development.
5. Supports REST API development.

Conclusion

Spring Boot makes Java application development easier through Spring Initializr, starter dependencies and auto-configuration. It helps developers create and run applications quickly with less configuration.

QUESTION 48

48. Explain the scope of a bean and Logger?

48. Explain the Scope of a Bean and Logger.

1. Bean Scope in Spring

Bean Scope defines the lifecycle and visibility of a Spring Bean. It determines how many instances of a bean are created and how long they remain available in the Spring IoC Container.

Spring supports six main bean scopes:

1. **Singleton:** Only one instance of a bean is created per Spring IoC Container. It is the default scope.
2. **Prototype:** A new bean instance is created every time the bean is requested from the container.
3. **Request:** A new bean instance is created for each HTTP request.
4. **Session:** One bean instance is created for each HTTP session.
5. **Application:** One bean instance is created per ServletContext and shared across the web application.
6. **WebSocket:** A separate bean instance is created for each WebSocket session.

Example:

```
@Component
@Scope("prototype")
public class Student {
}
```

Here, Spring creates a new Student bean whenever it is requested.

2. Logger in Spring Boot

Logger is a tool used to record application activities, events, warnings and errors during program execution.

Spring Boot commonly uses **SLF4J** as a logging API and **Logback** as its default logging implementation.

Uses of Logger

1. **Debugging:** Helps identify errors in the application.
2. **Monitoring:** Tracks application activities.
3. **Error Tracking:** Records exceptions and failures.
4. **Information Recording:** Displays important application events.

Example of Logger

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@RestController
public class StudentController {

    private static final Logger logger =
        LoggerFactory.getLogger(StudentController.class);

    @GetMapping("/student")
    public String getStudent() {

        logger.info("Student API called");
        logger.warn("Warning message");
        logger.error("Error occurred");

        return "Student Details";
    }
}
```

Logging Levels

Level	Purpose
TRACE	Detailed information
DEBUG	Debugging information
INFO	Normal application activities
WARN	Potential problems
ERROR	Serious errors

Conclusion

Bean Scope controls the lifecycle and number of Spring Bean instances, whereas Logger records application activities and errors to help developers monitor and maintain applications.

QUESTION 49

49. Illustrate how to implement AOP advices in Spring.

49. Illustrate How to Implement AOP Advices in Spring.

AOP (Aspect-Oriented Programming) is a programming approach in Spring used to separate common tasks from the main business logic. These common tasks are called **Cross-Cutting Concerns**, such as logging, security and transaction management.

Advice defines the action performed by an aspect at a particular point during program execution.

1. Types of AOP Advice in Spring

Spring supports five main types of advice:

1. **Before Advice:** Executes before the target method runs.
2. **After Advice:** Executes after the target method finishes, whether successfully or exceptionally.
3. **After Returning Advice:** Executes only when the target method completes successfully.
4. **After Throwing Advice:** Executes when the target method throws an exception.
5. **Around Advice:** Executes before and after the target method and can control its execution.

2. Steps to Implement AOP

Step 1: Add Dependency

Add the Spring AOP dependency in the `pom.xml` file.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Step 2: Create Service Class

```
@Service
public class StudentService {

    public void display() {
        System.out.println("Student Details");
    }
}
```

Step 3: Create Aspect Class

```java
@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* com.example.*.*(..))")
    public void beforeAdvice() {
        System.out.println("Before Method Execution");
    }

    @After("execution(* com.example.*.*(..))")
    public void afterAdvice() {
        System.out.println("After Method Execution");
    }

    @AfterReturning("execution(* com.example.*.*(..))")
    public void afterReturningAdvice() {
        System.out.println("Method Completed Successfully");
    }

    @AfterThrowing("execution(* com.example.*.*(..))")
    public void afterThrowingAdvice() {
        System.out.println("Exception Occurred");
    }
}
```

3. Explanation

- `@Aspect` identifies the class as an aspect.
- `@Component` registers it as a Spring Bean.
- `@Before` executes before the target method.
- `@After` executes after the target method.
- `@AfterReturning` executes after successful completion.
- `@AfterThrowing` executes when an exception occurs.

4. Benefits

1. Reduces code duplication.
2. Improves code reusability.
3. Separates business logic from common tasks.
4. Makes applications easier to maintain.

Conclusion

AOP advice helps execute common functionality at specific points during method execution. It makes Spring applications modular, reusable and maintainable.

QUESTION 50

50. Explain about Spring AOP.

50. Explain About Spring AOP.

Spring AOP (Aspect-Oriented Programming) is a programming approach in the Spring Framework used to separate common tasks from the main business logic of an application.

These common tasks are called **Cross-Cutting Concerns**, such as logging, security, transaction management and exception handling.

Spring AOP helps developers manage these tasks separately without repeatedly writing the same code in different classes.

1. Important Terminologies of Spring AOP

1. **Aspect:** A class that contains common functionality, such as logging or security.
2. **Join Point:** A point during program execution where an aspect can be applied, such as method execution.
3. **Advice:** The action performed by an aspect at a particular join point.
4. **Pointcut:** An expression that identifies where advice should be executed.
5. **Target Object:** The object whose method execution is affected by an aspect.
6. **Weaving:** The process of connecting aspects with application code.

2. Types of Advice in Spring AOP

1. **Before Advice:** Executes before a method.
2. **After Advice:** Executes after a method finishes.
3. **After Returning Advice:** Executes when a method completes successfully.
4. **After Throwing Advice:** Executes when a method throws an exception.
5. **Around Advice:** Executes before and after a method.

3. Example of Spring AOP

```
@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* com.example.service.*.*(..))")
    public void logBefore() {
        System.out.println("Method is executing");
    }
}
```

Explanation:

- `@Aspect` identifies the class as an aspect.
- `@Component` registers the class as a Spring Bean.
- `@Before` executes advice before the target method.
- `execution()` defines which methods should be intercepted.

4. Benefits of Spring AOP

1. **Code Reusability:** Common functionality can be reused.
2. **Loose Coupling:** Separates business logic from common tasks.
3. **Easy Maintenance:** Common functionality can be modified centrally.
4. **Reduced Code Duplication:** Avoids repeating logging and security code.
5. **Better Readability:** Keeps business logic clean and simple.

5. Applications of Spring AOP

- Logging
- Security
- Transaction Management
- Exception Handling
- Performance Monitoring

Conclusion

Spring AOP separates cross-cutting concerns from business logic, making applications more modular, reusable and maintainable.

QUESTION 51

51. Explain the Limitations of JDBC API and Query approaches.

51. Explain the Limitations of JDBC API and Query Approaches.

JDBC (Java Database Connectivity) is a Java API used to connect Java applications with databases and perform operations such as INSERT, UPDATE, DELETE and SELECT.

Although JDBC provides database connectivity, it has several limitations related to coding, resource management and query handling.

1. Limitations of JDBC API

1. **Boilerplate Code:** JDBC requires lengthy code for creating connections, statements and processing results.
2. **Manual Resource Management:** Developers must manually close connections, statements and result sets to prevent resource leaks.
3. **SQL Dependency:** Developers need to write SQL queries manually for database operations.
4. **Exception Handling:** JDBC requires handling `SQLException` explicitly.
5. **Object Mapping:** Developers must manually convert database records into Java objects.
6. **Difficult Maintenance:** Repeated database code makes large applications difficult to maintain.

2. Limitations of Traditional Query Approaches

1. **Complex Queries:** Writing complex SQL queries requires more effort and knowledge of SQL.
2. **Query Repetition:** Similar queries may need to be written repeatedly in different parts of an application.
3. **Database Dependency:** Queries may need modifications when changing database systems.
4. **Manual Result Processing:** Developers must process query results and map them to Java objects.
5. **Maintenance Difficulty:** Changes in database structure may require modifications to multiple queries.

3. How Spring Data JPA Solves These Limitations

Spring Data JPA simplifies database operations using repository interfaces and automatic query generation.

Example:

```java id="8vd8up" public interface EmployeeRepository extends JpaRepository {

```
List<Employee> findByName(String name);
```

} ```

Here, Spring Data JPA automatically generates the required query based on the method name.

## 4. Advantages of Spring Data JPA

1. Reduces boilerplate code.
2. Supports automatic object mapping.
3. Simplifies CRUD operations.
4. Provides pagination and sorting.
5. Improves code reusability and maintenance.
```

Conclusion

Traditional JDBC requires manual SQL queries, resource management and object mapping. Spring Data JPA is preferred because it simplifies database operations, reduces coding complexity and improves application maintainability.

QUESTION 52

52. Demonstrate how to implement pagination using a programming framework.

52. Demonstrate How to Implement Pagination Using a Programming Framework.

Pagination is a technique used to divide a large amount of database records into smaller pages. Instead of retrieving all records at once, it fetches a limited number of records per page.

In **Spring Data JPA**, pagination is implemented using `Pageable`, `PageRequest` and `Page`.

1. Steps to Implement Pagination

Step 1: Create Entity Class

```
@Entity
public class User {
    @Id
    private int id;
    private String name;
    private String email;

    // Getters and Setters
}
```

Step 2: Create Repository Interface

```
public interface UserRepository
        extends JpaRepository<User, Integer> {
}
```

Here, `JpaRepository` provides built-in pagination methods.

Step 3: Implement Pagination in Service Class

```
@Service
public class UserService {

    @Autowired
    private UserRepository repository;

    public Page<User> getUsers(
            int page, int size) {

        Pageable pageable =
            PageRequest.of(page, size);

        return repository.findAll(pageable);
    }
}
```

Step 4: Create REST Controller

```
@RestController
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserService service;

    @GetMapping
    public Page<User> getUsers(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "5") int size) {

        return service.getUsers(page, size);
    }
}
```

2. Explanation

1. **Pageable:** Defines the page number and page size.
2. **PageRequest:** Creates a pagination request.
3. **Page:** Stores the requested records and pagination details.
4. **findAll(pageable):** Retrieves records according to the pagination settings.

3. Testing Using Postman

Enter the following URL:

```
http://localhost:8080/users?page=0&size=5
```

- `page=0`: Retrieves the first page.
- `size=5`: Displays five records per page.

For the second page:

```
http://localhost:8080/users?page=1&size=5
```

4. Advantages of Pagination

1. Reduces unnecessary data retrieval.
2. Improves application performance.
3. Reduces memory consumption.
4. Makes large datasets easier to manage.

Conclusion

Spring Data JPA provides `Pageable`, `PageRequest` and `Page` to implement pagination efficiently. It helps retrieve database records in smaller pages without writing manual SQL queries.

QUESTION 53

53. Explain the Update Operation in Spring Data JPA.

53. Explain the Update Operation in Spring Data JPA.

Update Operation in Spring Data JPA is used to modify existing records in a database. It allows developers to change the values of an entity without writing complex SQL UPDATE queries manually.

Spring Data JPA uses the `save()` method and the `@Transactional` annotation to perform update operations.

1. Steps to Implement Update Operation

Step 1: Create Entity Class

```
@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    private String name;
    private double salary;

    // Getters and Setters
}
```

Step 2: Create Repository Interface

```
public interface EmployeeRepository
        extends JpaRepository<Employee, Integer> {
}
```

Here, `JpaRepository` provides the built-in `save()` and `findById()` methods.

Step 3: Create Service Class

```
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repository;

    @Transactional
    public Employee updateEmployee(
            int id, String name, double salary) {

        Employee emp = repository.findById(id)
            .orElseThrow(() ->
                new RuntimeException("Employee not found"));

        emp.setName(name);
        emp.setSalary(salary);

        return repository.save(emp);
    }
}
```

Step 4: Create REST Controller

```
@RestController
@RequestMapping("/employees")
public class EmployeeController {

    @Autowired
    private EmployeeService service;

    @PutMapping("/{id}")
    public Employee updateEmployee(
            @PathVariable int id,
            @RequestBody Employee emp) {

        return service.updateEmployee(
            id, emp.getName(), emp.getSalary());
    }
}
```

2. Explanation of Update Operation

1. **findById():** Searches for the employee using the given ID.
2. **orElseThrow():** Generates an exception if the employee does not exist.
3. **setName():** Updates the employee's name.
4. **setSalary():** Updates the employee's salary.
5. **save():** Saves the modified entity.
6. **@Transactional:** Ensures that the database operation runs within a transaction.

3. Testing Using Postman

Send a PUT request:

```
PUT http://localhost:8080/employees/1
```

Request Body (JSON):

```
{
  "name": "Rahul",
  "salary": 45000
}
```

The employee with ID 1 will be updated with the new name and salary.

4. Advantages

1. Simplifies database update operations.
2. Reduces manual SQL coding.
3. Maintains data consistency through transactions.
4. Supports easy integration with REST APIs.

Conclusion

Spring Data JPA makes updating database records simple using `findById()`, entity setter methods and `save()`. The `@Transactional` annotation helps maintain transaction consistency during the update operation.

QUESTION 54

54. Discuss about Named Queries and Query.

54. Discuss Named Queries and @Query in Spring Data JPA.

Named Queries and **@Query** are two approaches used in Spring Data JPA to execute custom database queries. They help developers retrieve, update or delete database records according to application requirements.

1. Named Queries

A **Named Query** is a predefined query that is given a specific name and declared using the `@NamedQuery` annotation in an entity class.

It is generally written using JPQL (Java Persistence Query Language).

Example:

```
@Entity
@NamedQuery(
    name = "Employee.findByName",
    query = "SELECT e FROM Employee e WHERE e.name = :name"
)
public class Employee {

    @Id
    private int id;

    private String name;
    private double salary;

    // Getters and Setters
}
```

Repository Interface:

```
public interface EmployeeRepository
        extends JpaRepository<Employee, Integer> {

    List<Employee> findByName(@Param("name") String name);
}
```

Explanation:

1. `@NamedQuery` defines a query with a unique name.
2. `name` identifies the query.
3. `query` contains the JPQL statement.
4. `:name` represents a named parameter.
5. Spring Data JPA can automatically find the named query using the entity and repository method names.

2. @Query Annotation

The `@Query` annotation is used to define a custom query directly inside a repository interface.

It supports JPQL and native SQL queries.

Example:

```
public interface EmployeeRepository
        extends JpaRepository<Employee, Integer> {

    @Query("SELECT e FROM Employee e WHERE e.salary > :salary")
    List<Employee> findHighSalaryEmployees(
        @Param("salary") double salary);
}
```

Explanation:

1. `@Query` defines a custom query.
2. JPQL uses entity and field names.
3. `:salary` represents a named parameter.
4. `@Param` connects the method parameter with the query parameter.
5. The method returns employees whose salary is greater than the given amount.

Native SQL Example:

```
@Query(
    value = "SELECT * FROM employee WHERE salary > :salary",
    nativeQuery = true
)
List<Employee> findEmployeesBySalary(
    @Param("salary") double salary);
```

3. Difference Between Named Queries and @Query

Named Queries	@Query
Defined using `@NamedQuery`.	Defined using `@Query`.
Usually declared in an entity class or XML.	Declared in a repository interface.
Have a specific query name.	Written directly above repository methods.
Can be reused by referencing the named query.	Usually associated with a particular repository method.
Separates query definition from repository method.	Keeps query definition close to the method.

Conclusion

Named Queries and `@Query` provide ways to execute custom database queries in Spring Data JPA. Named Queries are useful for predefined queries, whereas `@Query` is convenient for writing custom queries directly inside repository interfaces.

QUESTION 55

55. Illustrate SOA-Service Oriented Architecture.

55. Illustrate SOA – Service Oriented Architecture.

SOA (Service Oriented Architecture) is a software architecture in which an application is developed using independent and reusable services. Each service performs a specific business function and communicates with other services through standard communication protocols.

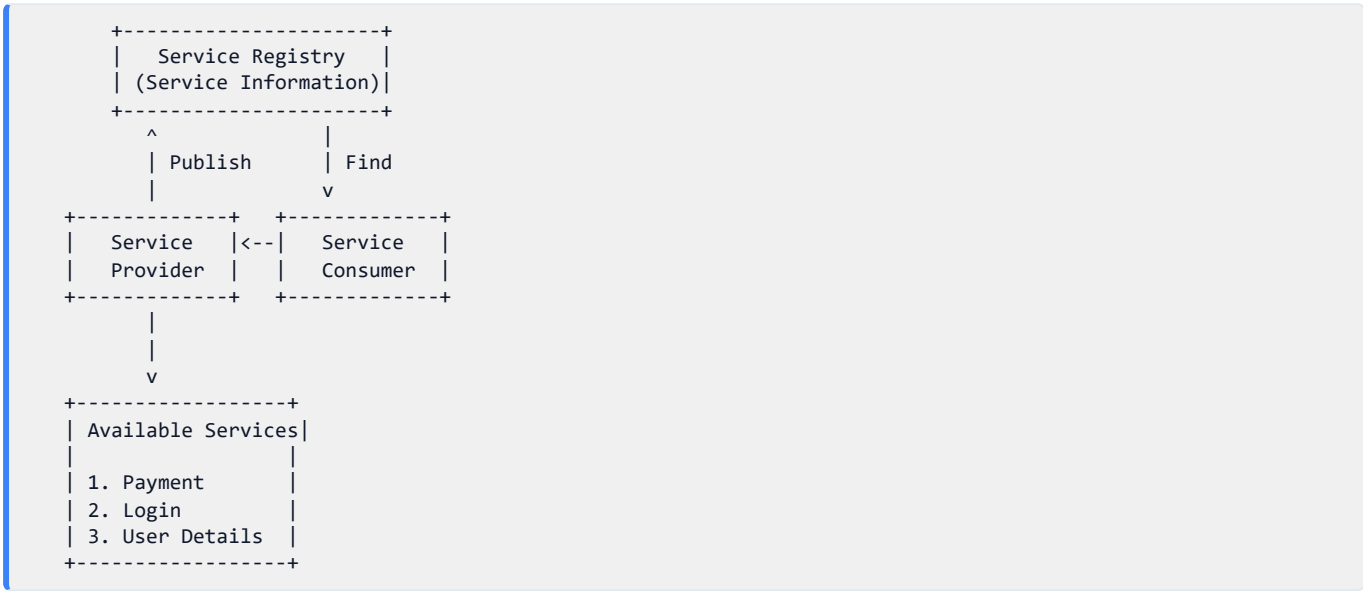
SOA helps different applications work together, even if they are developed using different programming languages or technologies.

1. Main Components of SOA

SOA mainly consists of three important components:

1. **Service Provider:** Creates and provides services to other applications. It publishes service information so that consumers can use it.
2. **Service Consumer:** A user or application that requests and uses services provided by the service provider.
3. **Service Registry:** Stores information about available services. Consumers can search the registry to find suitable services.

2. Architecture Diagram of SOA



3. Working of SOA

SOA generally follows three main operations:

1. **Publish:** The service provider publishes service details in the service registry.
2. **Find:** The service consumer searches the registry to find the required service.
3. **Bind/Invoke:** The consumer connects to the provider and invokes the selected service to perform an operation.

4. Main Principles of SOA

1. **Loose Coupling:** Services work independently with minimum dependency on each other.
2. **Reusability:** A service can be reused by multiple applications.
3. **Interoperability:** Services developed using different technologies can communicate.
4. **Service Autonomy:** Each service manages its own functionality independently.
5. **Standardized Contract:** Services communicate through clearly defined interfaces.
6. **Discoverability:** Services can be identified and accessed through service descriptions.

5. Example of SOA

Consider an online shopping application. It uses different services:

- **Login Service:** Authenticates users.
- **Product Service:** Displays product details.
- **Payment Service:** Processes payments.
- **Delivery Service:** Manages product delivery.

All these services work together to complete an online purchase.

6. Advantages of SOA

1. Improves code reusability.
2. Reduces application development time.
3. Supports integration between different systems.
4. Makes applications easier to maintain.
5. Provides flexibility and scalability.

Conclusion

SOA is an architecture that divides applications into independent and reusable services. Its main components are Service Provider, Service Consumer and Service Registry. It improves flexibility, interoperability and reusability in distributed applications.

QUESTION 56

56. Explain various Types of Web Services.

56. Explain Various Types of Web Services.

Web Services are software components that allow two different applications to communicate and exchange data over a network. They use standard communication protocols and data formats to share information between different platforms and programming languages.

There are mainly two important types of Web Services:

1. SOAP Web Services

SOAP (Simple Object Access Protocol) is a protocol used for exchanging structured information between applications over a network.

Features:

1. Uses XML format for exchanging messages.
2. Follows a strict messaging structure.
3. Supports different transport protocols, such as HTTP and SMTP.
4. Provides security and transaction-related standards through WS-* specifications.
5. Uses WSDL (Web Services Description Language) to describe available services.

Example:

A banking application can use SOAP Web Services to exchange secure transaction information between different banking systems.

SOAP Message Structure:

```
<soap:Envelope>
    <soap:Header>
        <!-- Header Information -->
    </soap:Header>

    <soap:Body>
        <!-- Request or Response Data -->
    </soap:Body>
</soap:Envelope>
```

2. RESTful Web Services

REST (Representational State Transfer) is an architectural style used to develop Web Services. It allows applications to communicate using standard HTTP methods.

Features:

1. Uses HTTP methods such as GET, POST, PUT and DELETE.
2. Commonly uses JSON for data exchange, but also supports XML and other formats.
3. Follows a stateless communication approach.
4. Uses URLs to identify resources.

5. Simple, lightweight and suitable for modern web and mobile applications.

Example:

A student management application can use REST APIs to retrieve, add, update and delete student records.

REST API Example:

```
GET    /students       - Retrieve students
POST   /students       - Add a student
PUT    /students/1     - Update a student
DELETE /students/1     - Delete a student
```

3. Difference Between SOAP and REST

SOAP	REST
It is a protocol.	It is an architectural style.
Uses XML messages.	Supports JSON, XML and other formats.
Follows strict message rules.	Uses standard HTTP methods.
Supports WS-* standards.	Uses HTTP security mechanisms and other security approaches.
Generally more complex.	Generally simpler and lightweight.
Can use HTTP, SMTP and other transports.	Commonly uses HTTP/HTTPS.

4. Other Web Service Technologies

- **XML-RPC:** Uses XML to encode requests and responses and performs remote procedure calls.
- **JSON-RPC:** Uses JSON format for remote procedure calls between applications.

Conclusion

SOAP and REST are the two major approaches used in Web Services. SOAP provides a structured protocol with enterprise-level standards, whereas REST provides a simpler and flexible approach for developing modern Web APIs.

QUESTION 57

57. Show how to design and implement RESTful APIs for client-server communication.

57. Show How to Design and Implement RESTful APIs for Client-Server Communication.

RESTful API (Representational State Transfer Application Programming Interface) is used to enable communication between a client and a server over HTTP. It allows applications to exchange data using standard HTTP methods such as GET, POST, PUT and DELETE.

In Spring Boot, RESTful APIs can be implemented using annotations such as `@RestController`, `@GetMapping`, `@PostMapping`, `@PutMapping` and `@DeleteMapping`.

1. Basic Architecture

RESTful API follows a client-server architecture.

```
+------------------+
|   Client         |
| (Browser/Postman)|
+------------------+
         |
         | HTTP Request
         v
+------------------+
|   REST API       |
| (Spring Boot)    |
+------------------+
         |
         | Process Request
         v
+------------------+
|   Database       |
+------------------+
         |
         | Response Data
         v
+------------------+
|   Client         |
+------------------+
```

2. Steps to Implement RESTful API

Step 1: Create a Spring Boot Project

Use Spring Initializr and add these dependencies:

- Spring Web
- Spring Data JPA
- MySQL Driver (if using MySQL)

Step 2: Create Entity Class

```java
public class Student {
    private int id;
    private String name;

    // Getters and Setters
}
```

Step 3: Create REST Controller

```java
@RestController
@RequestMapping("/students")
public class StudentController {

    private List<Student> students = new ArrayList<>();

    // GET operation
    @GetMapping
    public List<Student> getStudents() {
        return students;
    }

    // POST operation
    @PostMapping
    public Student addStudent(@RequestBody Student s) {
        students.add(s);
        return s;
    }

    // PUT operation
    @PutMapping("/{id}")
    public String updateStudent(@PathVariable int id,
                                @RequestBody Student s) {
        for (Student student : students) {
            if (student.getId() == id) {
                student.setName(s.getName());
                return "Student updated";
            }
        }
        return "Student not found";
    }

    // DELETE operation
    @DeleteMapping("/{id}")
    public String deleteStudent(@PathVariable int id) {
        students.removeIf(s -> s.getId() == id);
        return "Student deleted";
    }
}
```

3. HTTP Methods Used in REST API

HTTP Method	Purpose
GET	Retrieve student records.
POST	Create a new student.
PUT	Update an existing student.
DELETE	Remove a student.

4. Testing Using Postman

The API can be tested using Postman.

- **GET:** `http://localhost:8080/students`
- **POST:** `http://localhost:8080/students`
- **PUT:** `http://localhost:8080/students/1`
- **DELETE:** `http://localhost:8080/students/1`

For POST, send JSON data:

```json
{
  "id": 1,
  "name": "Rahul"
}
```

5. Advantages

1. Supports communication between different applications.
2. Uses standard HTTP methods.
3. Supports JSON and XML data formats.
4. Provides platform independence.
5. Makes application integration easier.

Conclusion

RESTful APIs provide a simple and efficient method for client-server communication. Spring Boot simplifies their implementation using REST annotations and HTTP methods. These APIs are widely used in web, mobile and distributed applications.

QUESTION 58

58. Explain SOAP based Web Services.

58. Explain SOAP-Based Web Services.

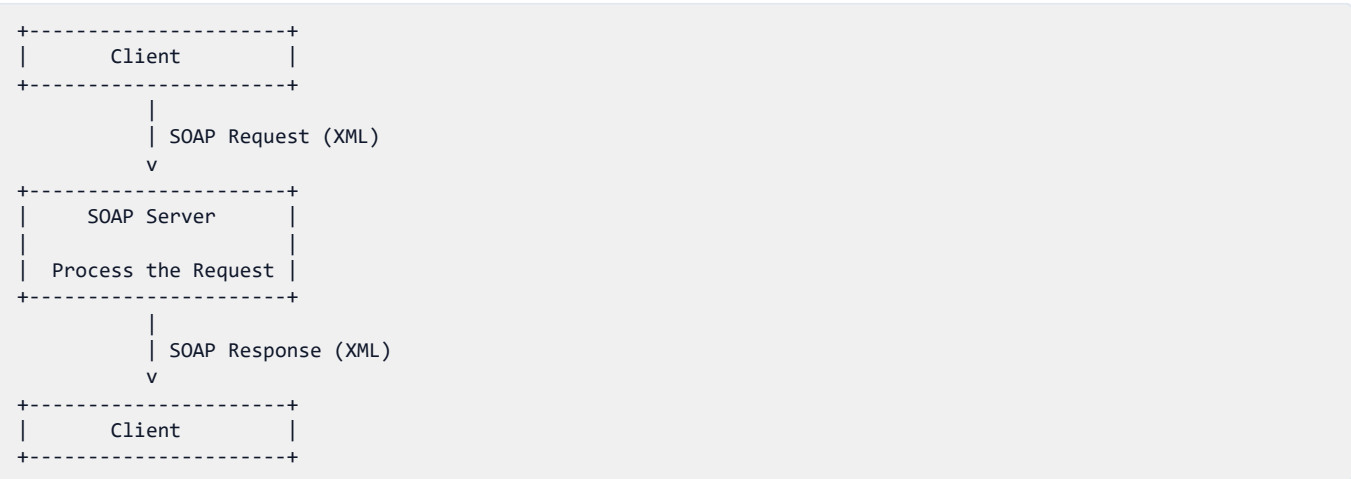
SOAP (Simple Object Access Protocol) is a messaging protocol used by applications to exchange structured information over a network. It allows different applications developed using different programming languages and platforms to communicate with each other.

SOAP Web Services commonly use **XML (Extensible Markup Language)** to exchange messages between client and server.

1. Main Components of SOAP

1. **SOAP Envelope:** Defines the beginning and end of a SOAP message. It contains the complete message.
2. **SOAP Header:** Contains additional information such as authentication, security and transaction details. It is optional.
3. **SOAP Body:** Contains the actual request or response data.
4. **SOAP Fault:** Provides error information when a SOAP message cannot be processed.

2. SOAP Architecture



3. SOAP Message Structure

A SOAP message follows a standard XML structure.

Example:

```
<soap:Envelope
 xmlns:soap="http://www.w3.org/2003/05/soap-envelope">

    <soap:Header>
        <!-- Additional Information -->
    </soap:Header>

    <soap:Body>
        <getStudent>
            <id>101</id>
        </getStudent>
    </soap:Body>

</soap:Envelope>
```

Explanation:

1. `Envelope` represents the complete SOAP message.
2. `Header` contains optional additional information.
3. `Body` contains the actual request data.
4. The server processes the request and sends an XML response.

4. Working of SOAP Web Services

1. **Client Request:** The client creates a SOAP request in XML format.
2. **Message Transmission:** The request is sent to the server using a supported transport protocol, commonly HTTP or HTTPS.
3. **Request Processing:** The server receives and processes the SOAP message.
4. **Response Generation:** The server creates a SOAP response in XML format.
5. **Client Receives Response:** The client receives and processes the response.

5. Important Technologies Used in SOAP

- **WSDL (Web Services Description Language):** Describes the web service, its operations and message formats.
- **UDDI (Universal Description, Discovery and Integration):** Provides a standard for publishing and discovering web services.
- **XML:** Used to structure SOAP messages.
- **HTTP/HTTPS:** Commonly used for message transmission.

6. Advantages of SOAP

1. Provides a standard messaging structure.
2. Supports communication between different platforms.
3. Supports security standards through WS-Security.
4. Supports enterprise-level transaction requirements.
5. Can work with multiple transport protocols.

7. Limitations of SOAP

1. XML messages can be lengthy.
2. Requires more processing than lightweight formats.
3. Implementation can be complex.
4. Generally requires more bandwidth.

Conclusion

SOAP-based Web Services provide a standardized way for applications to exchange structured XML messages. They are commonly used in enterprise systems where strict messaging standards, security and transaction support are important.

QUESTION 59

59. Explain about RequestBody and Response Entity and Parameter Injection.

59. Explain `@RequestBody`, `ResponseEntity` and Parameter Injection in Spring REST.

In Spring REST, `@RequestBody`, `ResponseEntity` and Parameter Injection are used to handle HTTP requests and responses. They help transfer data between the client and server.

1. @RequestBody

`@RequestBody` is an annotation used to read data from the body of an HTTP request and convert it into a Java object.

Example:

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @PostMapping
    public String addStudent(@RequestBody Student student) {
        return "Student added: " + student.getName();
    }
}
```

JSON Request:

```
{
  "id": 101,
  "name": "Rahul"
}
```

Explanation:

1. The client sends JSON data in the request body.
2. `@RequestBody` receives the JSON data.
3. Spring uses an `HttpMessageConverter` (commonly Jackson) to convert JSON into a Java object.
4. The controller accesses the object using its getter methods.

Use: It is mainly used in POST and PUT requests to receive structured data.

2. ResponseEntity

`ResponseEntity` is a Spring class used to control the complete HTTP response. It allows developers to return response data, HTTP status codes and headers.

Example:

```
@GetMapping("/{id}")
public ResponseEntity<Student> getStudent(
        @PathVariable int id) {

    Student student = new Student(id, "Rahul");

    return ResponseEntity.ok(student);
}
```

Explanation:

1. `ResponseEntity` represents the HTTP response.
2. `ok()` returns status code 200 (OK).
3. The student object is included in the response body.
4. Developers can also set headers and different status codes.

Example of Not Found Response:

```
return ResponseEntity.notFound().build();
```

It returns HTTP status code 404 (Not Found).

3. Parameter Injection

Parameter Injection means receiving values from an HTTP request directly into controller method parameters using Spring annotations.

Important annotations include:

A. @PathVariable

It extracts values from the URL path.

```
@GetMapping("/students/{id}")
public String getStudent(@PathVariable int id) {
    return "Student ID: " + id;
}
```

URL: `/students/101`

Output: `Student ID: 101`

B. @RequestParam

It extracts values from URL query parameters.

```
@GetMapping("/search")
public String searchStudent(
        @RequestParam String name) {
    return "Student Name: " + name;
}
```

URL: `/students/search?name=Rahul`

Output: `Student Name: Rahul`

4. Difference Between Them

Feature	@RequestBody	ResponseEntity	Parameter Injection
Purpose	Reads request body.	Controls HTTP response.	Receives request parameters.
Direction	Client to server.	Server to client.	Client to server.
Data source	JSON/XML body.	Response body, headers and status.	URL path or query parameters.
Common use	POST and PUT.	All HTTP response types.	GET and other requests.

Conclusion

`@RequestBody` receives structured data, `ResponseEntity` controls HTTP responses, and parameter injection extracts values from request URLs. These features make Spring REST API development easier and more flexible.

QUESTION 60

60. Apply Exception Handling techniques and Data Validation rules to develop a robust and error-free program.

60. Apply Exception Handling Techniques and Data Validation Rules to Develop a Robust and Error-Free Program.

Exception Handling and **Data Validation** are important features in Spring Boot REST applications. They help prevent invalid data, handle unexpected errors and provide meaningful responses to clients.

1. Exception Handling

Exception Handling is a mechanism used to handle runtime errors without stopping the entire application.

Spring Boot provides the following annotations:

1. **@ExceptionHandler:** Handles specific exceptions inside a controller or advice class.
2. **@ControllerAdvice:** Provides centralized exception handling across multiple controllers.
3. **@RestControllerAdvice:** Combines `@ControllerAdvice` and `@ResponseBody` to return error responses as JSON.
4. **ResponseEntity:** Returns an appropriate HTTP status code with an error message.

2. Implementing Exception Handling

Example: Global Exception Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class)
    public ResponseEntity<String> handleException(
            Exception ex) {

        return ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Something went wrong!");
    }
}
```

Explanation:

1. `@RestControllerAdvice` handles exceptions globally.
2. `@ExceptionHandler` identifies the exception to handle.
3. `ResponseEntity` sends an error message.
4. HTTP 500 indicates an internal server error.

3. Data Validation

Data Validation checks whether the input provided by users follows the required rules before processing or storing it.

Spring Boot supports validation using Jakarta Bean Validation annotations.

Common validation annotations:

Annotation	Purpose
`@NotNull`	Value must not be null.
`@NotBlank`	String must not be null or empty after trimming whitespace.
`@Size`	Defines minimum and maximum length.
`@Email`	Checks email format.
`@Min`	Defines minimum numeric value.
`@Max`	Defines maximum numeric value.

4. Implementing Data Validation

Step 1: Create Student Class

```
public class Student {

    @NotBlank(message = "Name is required")
    private String name;

    @Email(message = "Invalid email")
    private String email;

    @Min(value = 18, message = "Minimum age is 18")
    private int age;

    // Getters and Setters
}
```

Step 2: Apply Validation in Controller

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @PostMapping
    public String addStudent(
            @Valid @RequestBody Student student) {

        return "Student added successfully";
    }
}
```

Add the `spring-boot-starter-validation` dependency to enable validation.

5. Benefits

1. Prevents invalid data from entering the database.
2. Improves application reliability.
3. Provides meaningful error messages.
4. Makes debugging easier.
5. Improves security and data consistency.

Conclusion

Exception Handling and Data Validation make Spring Boot applications more reliable. Exception handling manages unexpected errors, while validation checks user input before processing. Together, they help develop robust and maintainable REST applications.

QUESTION 61

61. Explain steps to Create a REST Client.

61. Explain the Steps to Create a REST Client.

A **REST Client** is an application or software component that communicates with a RESTful Web Service through HTTP requests. It sends requests to a REST API and receives responses from the server in formats such as JSON or XML.

In Spring Boot, a REST Client can be created using `RestTemplate` or `WebClient`.

1. Steps to Create a REST Client Using RestTemplate

Step 1: Create a Spring Boot Project

Open Spring Initializr and create a project with the following dependency:

- Spring Web

Step 2: Configure RestTemplate

Create a configuration class to provide a `RestTemplate` bean.

```
@Configuration
public class AppConfig {

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

Step 3: Create a REST Client Service

Use `RestTemplate` to communicate with an external REST API.

```
@Service
public class ApiClient {

    @Autowired
    private RestTemplate restTemplate;

    public String getData() {

        String url =
            "https://jsonplaceholder.typicode.com/posts/1";

        return restTemplate.getForObject(
            url, String.class);
    }
}
```

Step 4: Create a Controller

```
@RestController
public class ClientController {

    @Autowired
    private ApiClient apiClient;

    @GetMapping("/client/data")
    public String getData() {
        return apiClient.getData();
    }
}
```

Step 5: Run and Test the Application

Start the Spring Boot application and open Postman.

Send the following GET request:

```
http://localhost:8080/client/data
```

The REST Client sends a request to the external API and returns its response to the user.

2. HTTP Methods Used by REST Clients

HTTP Method	Purpose
GET	Retrieves data from the server.
POST	Sends new data to the server.
PUT	Updates existing data.
DELETE	Deletes existing data.

3. Advantages of REST Client

1. **Easy Communication:** Connects different applications through HTTP.
2. **Platform Independent:** Supports communication between different programming languages.
3. **Data Exchange:** Supports JSON and XML formats.
4. **Code Reusability:** Common API communication logic can be reused.
5. **Integration:** Helps applications communicate with external services.

4. Other REST Client Technologies

- **RestTemplate:** Traditional synchronous REST client provided by Spring.
- **WebClient:** Modern, non-blocking REST client provided by Spring WebFlux.
- **HTTP Interface Client:** Allows developers to define HTTP service methods using Java interfaces.

Conclusion

A REST Client allows one application to communicate with another application through REST APIs. Spring Boot provides `RestTemplate` and `WebClient` to send HTTP requests and receive responses efficiently.

QUESTION 62

62. Discuss about Securing Spring REST endpoints.

62. Discuss About Securing Spring REST Endpoints.

Securing Spring REST Endpoints means protecting REST APIs from unauthorized access, data theft and unwanted modifications. It ensures that only authenticated and authorized users can access specific resources.

Spring Boot provides **Spring Security** to protect REST endpoints using authentication, authorization and other security mechanisms.

1. Important Security Techniques

1. **Authentication:** Verifies the identity of a user using credentials such as username and password.
2. **Authorization:** Controls which resources or operations an authenticated user can access.
3. **HTTPS:** Encrypts communication between the client and server to protect sensitive information.

4. **JWT (JSON Web Token):** Uses digitally signed tokens to authenticate API requests without maintaining a traditional server-side session.
5. **Role-Based Access Control:** Gives permissions based on user roles such as ADMIN and USER.
6. **CORS Configuration:** Controls which external origins are allowed to access REST APIs through browsers.

2. Steps to Secure REST Endpoints Using Spring Security

Step 1: Add Dependency

Add the following dependency to the Maven `pom.xml` file:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Step 2: Create Security Configuration

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(
            HttpSecurity http) throws Exception {

        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}
```

Explanation:

1. `@EnableWebSecurity` enables Spring Security configuration.
2. `permitAll()` allows public access without authentication.
3. `hasRole("ADMIN")` allows only users with the ADMIN role.
4. `authenticated()` requires authentication for other endpoints.
5. `httpBasic()` enables HTTP Basic Authentication.

Note: CSRF protection should not be disabled blindly. This example assumes a stateless API using non-cookie-based authentication. For browser-based session authentication, appropriate CSRF protection is required.

3. Testing Using Postman

1. Open Postman.
2. Enter the REST API URL.
3. Select the Authorization tab.
4. Choose Basic Auth.
5. Enter the configured username and password.
6. Send the request.

If authentication succeeds and the user has permission, the server returns the requested data.

4. Advantages of Securing REST Endpoints

1. Prevents unauthorized access.
2. Protects confidential information.
3. Provides role-based permissions.
4. Improves application security.
5. Protects communication through HTTPS.

Conclusion

Spring Security helps protect REST endpoints through authentication, authorization and role-based access control. By implementing suitable security configurations, developers can build secure and reliable Spring Boot REST applications.

If you want, I can:

- Explain detailed example of exception handling in Spring REST
- Show step-by-step how to create a Spring REST client with WebClient
- Demonstrate securing REST endpoints using JWT authentication in Spring Security